VECTOR

Boot Manager

Technical Reference

Version 2.1.1

Authors	Marco Wierer, Marco Riedl, Matteo Trovo
Status	Released

Document Information

History

Author	Date	Version	Remarks
Marco Wierer, Marco Riedl	2019-02-15	1.0.0	Initial version
Marco Riedl	2019-07-09	1.1.0	> General rework of open points from version 1.0.0 > Updated note in chapter 5.2.3.4 > Added API description for FblBmHdr (6.2) and FblSb (6.2.2.1) component > Added chapter 5.1.2 > Updated generated file list in 4.4.1.2
Marco Riedl	2019-10-21	1.1.1	> Fixed typo in Table 3-1 > Rephrased note box in chapter 3.3.4.4
Marco Riedl	2019-12-13	2.0.0	> Removed FblSb specific part. > Added information regarding presence pattern in the Bootloader (chapter 4.2.1).
Marco Riedl	2020-01-17	2.1.0	> Adapted chapter 5.1.1
Matteo Trovo	2020-03-12	2.1.1	> Add caution box in chapter 5.2.1.1

Reference Documents

No.	Source	Title	Version
[1]	Vector	Technical Reference – MICROSAR Memory Mapping	1.3.0
[2]	Vector	Technical Reference – MICROSAR CSM	
[3]	Vector	Technical Reference – Flash Bootloader OEM	
[4]	Vector	Technical Reference – Flash Bootloader Updater	1.2.0
[5]	HIS	SHE Functional Description	1.1 Rev.:439
[6]	Vector	Technical Reference – FblLib SecureBoot	

Caution
We have configured the programs in accordance with your specifications in the questionnaire. Whereas the programs do support other configurations than the one specified in your questionnaire, Vector´s release of the programs delivered to your company is expressly restricted to the configuration you have specified in the questionnaire.

© 2020 Vector Informatik GmbH

Contents

1 Component History 8
1.1 Version 1.00.00 8
1.2 Version 2.00.00 8
1.3 Version 3.00.00 8
1.3.1 What is new? 8
1.4 Version 4.00.00 8
1.5 Version 5.00.00 8

2 Introduction 9
2.1 Startup Sequence 9

3 Functional Description 11
3.1 Features 11
3.2 General Concept 11
3.2.1 Boot Sequence 11
3.2.2 FblBmHdrHeader Concept 13
3.2.3 Memory Layout 13
3.3 Specific Features 14
3.3.1 Failsafe Flash Bootloader Updater 14
3.3.2 Stay In Boot (Force Boot Mode) 14
3.3.3 Reprogrammable Logical Block Table 15
3.3.4 Secure Boot 15

4 Integration 21
4.1 Boot Manager 21
4.1.1 Flash Block Table 21
4.2 Flash Bootloader 21
4.2.1 Presence Pattern 21
4.2.2 Flash Block Table 21
4.2.3 Logical Block Table 21
4.3 Application 22
4.4 Scope of Delivery 22
4.4.1 FblBm 22
4.4.2 FblBmHdr 22

5 Configuration 24
5.1 FblBm 25
5.1.1 FblBm | FblBmGeneral 25
5.1.2 FblBm | FblBmBootOrder 27

5.2 FblBmHdr 29
5.2.1 FblBmHdr | FblBmHdrGeneral 29
5.2.2 FblBmHdr | FblBmHdrTargets 30
5.2.3 FblBmHdr | FblBmHdrHeader 30
5.3 Fbl 33
5.3.1 Fbl | FblGeneral 33
5.4 Project Settings 34
5.4.1 Vector vHSM 34

6 API Description 36
6.1 Overwriting API Callout Functions 36
6.1.1 Overview of Callout Functions (FBLBM_CALLOUT_) 36
6.2 Overview of FblBmHdr Functions 51
6.2.1 Core functions 51
6.2.2 Callback functions 54
6.3 Data Types 55
6.3.1 tFblBmHeader 55
6.3.2 tFblBmHdrHeader 56
6.3.3 tFblBmBlockInfo 57

7 Glossary and Abbreviations 58
7.1 Glossary 58
7.2 Abbreviations 58

8 Contact 60

Illustrations

Figure 2-1 Simplified startup sequence 10
Figure 3-1 Boot Sequence 12
Figure 3-2 Memory Layout and Access Direction 14
Figure 3-3 Sequence - Stay In Boot 15
Figure 3-4 Chain of trust concept 16
Figure 3-5 Sequence – Key Initialization 17
Figure 3-6 M1M2M3 key update protocol 18
Figure 3-7 M1M2M3 key update policy 19
Figure 3-8 Sequence – MAC Initialization 20
Figure 5-1 DaVinci Configurator: FblBmGeneral Configuration 25
Figure 5-2 DaVinci Configurator: FblBmBootOrder 27
Figure 5-3 DaVinci Configurator: FblBmBootTargets 28
Figure 5-4 DaVinci Configurator: FblBmDependencyTargets 29
Figure 5-5 DaVinci Configurator: FblBmHdrGeneral Configuration 29
Figure 5-6 DaVinci Configurator: FblBmHdrTargets Configuration 30
Figure 5-7 DaVinci Configurator: FblBmHdrHeader Configuration 30
Figure 5-8 DaVinci Configurator: FblBmHdrVerificationRanges Configuration 32
Figure 5-9 DaVinci Configurator: FblBmHdrAuthenticationHeader Configuration 33
Figure 5-10 DaVinci Configurator: FblGeneral Configuration 33
Figure 5-11 DaVinci Configurator: FblLogicalBlocks Configuration 34
Figure 5-12 DaVinci Configurator: Add additional Module Definitions 34
Figure 5-13 DaVinci Configurator: Configure destination of the Pre-Config file 35

Tables

Table 3-1 Advantages and disadvantages of different key initialization solutions 17
Table 4-1 Static files of FblBm 22
Table 4-2 Generated files of FblBm 22
Table 4-3 Static files of FblBmHdr 23
Table 4-4 Generated files of FblBmHdr 23
Table 6-1 Callout Functions Overview 37
Table 6-2 API Description: FBLBM_CALLOUT_RESET 37
Table 6-3 API Description: FBLBM_CALLOUT_POWER_ON_PRE_HW 38
Table 6-4 API Description: FBLBM_CALLOUT_POWER_ON_PRE_OEM 38
Table 6-5 API Description: FBLBM_CALLOUT_POWER_ON_PRE 39
Table 6-6 API Description: FBLBM_CALLOUT_POWER_ON_POST_HW 39
Table 6-7 API Description: FBLBM_CALLOUT_POWER_ON_POST_OEM 40
Table 6-8 API Description: FBLBM_CALLOUT_POWER_ON_POST 40
Table 6-9 API Description: FBLBM_CALLOUT_INIT_PRE_HW 41
Table 6-10 API Description: FBLBM_CALLOUT_INIT_PRE_OEM 41
Table 6-11 API Description: FBLBM_CALLOUT_INIT_PRE 42
Table 6-12 API Description: FBLBM_CALLOUT_DEINIT_PRE_HW 42
Table 6-13 API Description: FBLBM_CALLOUT_DEINIT_PRE_OEM 43
Table 6-14 API Description: FBLBM_CALLOUT_DEINIT_PRE 43
Table 6-15 API Description: FBLBM_CALLOUT_DEINIT_POST_HW 44
Table 6-16 API Description: FBLBM_CALLOUT_DEINIT_POST_OEM 44
Table 6-17 API Description: FBLBM_CALLOUT_DEINIT_POST 45
Table 6-18 API Description: FBLBM_CALLOUT_CHECK_TARGET_VALIDITY 45
Table 6-19 API Description: FBLBM_CALLOUT_IS_VALIDBLOCK 46
Table 6-20 API Description: FBLBM_CALLOUT_CALL_FBL 47
Table 6-21 API Description: FBLBM_CALLOUT_CALL_TARGET 47

Table 6-22 API Description: FBLBM_CALLOUT_CALL_FBL_INIT_KEYS 47
Table 6-23 API Description: FBLBM_CALLOUT_CALL_FBL_INIT_CMAC 48
Table 6-24 API Description: FBLBM_CALLOUT_GET_KEYEMPTY 48
Table 6-25 API Description: FBLBM_CALLOUT_GET_FBL_CMACERASED 49
Table 6-26 API Description: FBLBM_CALLOUT_SEARCH_FAILSAFE_UPDATER_INIT 49
Table 6-27 API Description: FBLBM_CALLOUT_SEARCH_FAILSAFE_UPDATER_HEADER_ADDRESS 50
Table 6-28 API Description: FBLBM_CALLOUT_SEARCH_FAILSAFE_UPDATER_NEXT 50
Table 6-29 API Description: FBLBM_CALLOUT_FATAL_ERROR 51
Table 6-30 FblBmHeader structure definition (tFblBmHeader) 56
Table 6-31 FblBmHdrHeader structure definition (tFblBmHdrHeader) for FblBmHdrVerificationList 56
Table 6-32 FblBmHdrHeader structure definition (tFblBmHdrHeader) for FblBmHdrAuthenticationHeader 57
Table 6-33 FblBmBlockInfo structure definition (tFblBmBlockInfo) 57

1 Component History

The changes to this component are shown in the following.

1.1 Version 1.00.00

Initial version.

1.2 Version 2.00.00

Introduced general check list and boot target concept.

1.3 Version 3.00.00

Added support for secure boot option.

1.3.1 What is new?

- Secure boot support added
- DaVinci Configurator support added

1.4 Version 4.00.00

- vBaseEnv support added

1.5 Version 5.00.00

- Different SecureBoot variants added
  - She: Already supported
  - vHsm: Support of SecureBoot groups
  - 3rdParty: Support of Ghs and Escrypt module

2 Introduction

This document covers the Boot Manager software component. The Boot Manager is a software component that is started right after the initialization of the micro controller. It performs the following tasks:

- ECU initialization (optional/partial):
  - Hardware
  - OEM
- Selecting a boot target, e.g. Flash Bootloader, Flash Bootloader Updater or application
- Checking the validity and in case of secure boot also checking the integrity and authenticity of a boot target



Note
It is recommended to perform hardware initializations, which improve the startup time of the Boot Manager (e.g. PLL), other hardware initializations have to be done by the Bootloader/application (e.g. CAN, GPIO).

2.1 Startup Sequence

Figure 2-1 shows the simplified startup sequence.

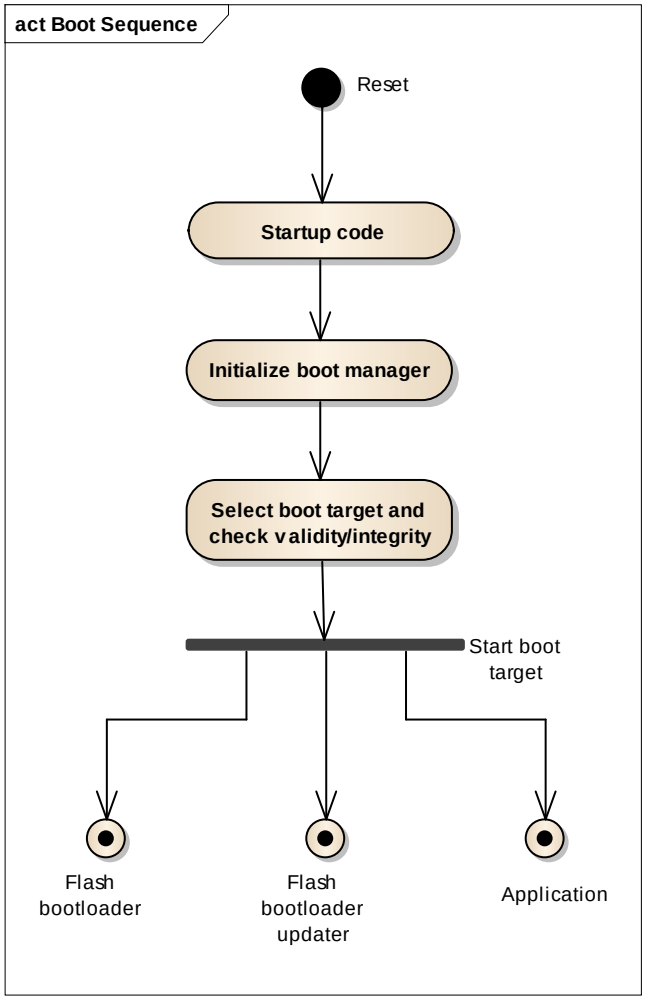


Figure 2-1 Simplified startup sequence

3 Functional Description

The Boot Manager controls the ECU booting process by deciding which “target” (Application, Flash Bootloader, Flash Bootloader Updater) will be started during ECU startup.

3.1 Features

The Boot Manager has the following general features:

- Reduced integration effort due to tight integration with Flash Bootloader.
- Great flexibility of user configuration and implementation.
- Secure Boot:
  - Simplified integration of secure boot components.
- Flash Bootloader Updater:
  - Hardware independent reset safe updates.
  - Fewer dependencies between Updater and Bootloader.
- Non Standalone Mode:
  - Can be integrated (compiled and linked) with the Flash Bootloader

3.2 General Concept

The Boot Manager uses the logical block table of the Flash Bootloader to allow a flexible configuration of the memory layout. Each logical block has a FblBmHdrHeader structure. The FblBmHdrHeader is referenced by the LBT and specifies what the logical block is (further known also as “target”).

The Boot Manager checks each logical block’s FblBmHdrHeader and decides whether the target needs to be started or not, based on the target order list configured in the FblBmBootOrder, see section 5.1.2.

3.2.1 Boot Sequence

The Boot Manager follows a specific state machine to start a valid target (e.g. bootloader, application, bootloader updater). This is illustrated in Figure 3-1:

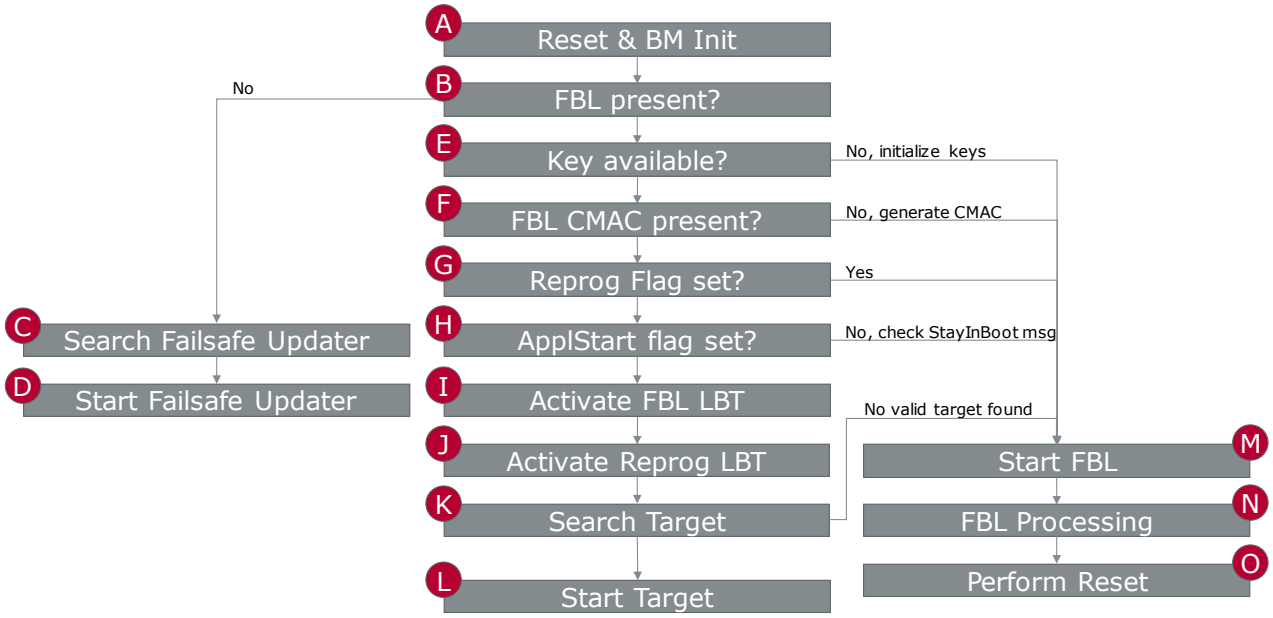


Figure 3-1 Boot Sequence

Each step of the Boot Sequence is described in detail:

A. Perform some basic initialization required for the boot sequence, e.g. hardware (PLL, Timer).

B. Check if the bootloader is present by checking the block validity. If this check fails go to Step C, otherwise Step E.

C. [Failsafe Updater] Search in the memory for an FblBmHdrHeader of a Bootloader Updater. For more information please refer to section 3.3.1.

D. [Failsafe Updater] In case the Boot Manager found a valid FblBmHdrHeader of a Bootloader Updater then the Boot Manager will start it.

E. [Secure Boot::Initialize Keys] The Boot Manager checks if the keys are already programmed in the hardware. If this is the case the Boot Manager goes to Step F, otherwise it continues with Step M. For more information please refer to section 3.3.4.3.

F. [Secure Boot::Initialize MAC Of Bootloader] The Boot Manager checks if the MAC of the Bootloader is present. If the MAC is not present the Boot Manager goes to Step M, otherwise to Step G. For more information please refer to section 3.3.4.4.

G. Check if a reprogramming request is present. If this is the case, then the Boot Manager goes to Step M, otherwise continues with Step H.

H. [Stay In Boot] Check if the ApplStart flag is set. If this is the case the Boot Manager continues with Step I. Otherwise, it will go to Step M. For more information please refer to section 3.3.2.

I. Activate the Logical Block Table of the Bootloader.

J. [Reprogrammable LBT] Activate the Reprogrammable Logical Block Table. For more information please refer to section 3.3.3.

K. The Boot Manager searches for a valid Boot Target. If a Boot Target and all related Dependency Targets are valid, then the Boot Manager will go to Step L. Otherwise to Step M.

L. Start the target, which was determined as valid in Step K, by jumping to the entry address specified in the FblBmHdrHeader.

M. Jump to the Bootloader as specified in the entry address of FblBmHdrHeader.

N. Perform the necessary Bootloader steps, depending on

   Step E: Initialize the configured hardware keys with random values, see section 3.3.4.3.

   Step F: Update the MAC of the Bootloader (e.g. generate and write the MAC in the flash), see section 3.3.4.4.

   Step G: The Bootloader waits for diagnostic requests to perform a download of an existing Logical Block (e.g. Application, Calibration).

   Step H: The Bootloader checks for a StayInBoot message in a defined time window. If the StayInBoot message was not received, the Bootloader sets the ApplFlag and performs a reset. Otherwise, if the message was received, the Bootloader stays in Bootloader and waits for diagnostic request to perform a download of a Logical Block (e.g. Application, Calibration).

   Step K: As no valid target was found, the Bootloader waits for diagnostic request to perform a download of a Logical Block (e.g. Application, Calibration).

O. The Bootloader performs a reset after the necessary steps as described in Step N are finished.

3.2.2 FblBmHdrHeader Concept

Each target contains a FblBmHdrHeader structure. For more details about the structure, please refer to section 6.3.2. The Boot Manager reads out and evaluates this structure to decide if this target is valid and if this target can be started.

3.2.3 Memory Layout

The Figure 3-2 shows the memory layout with the different headers and structures. The arrow indicates the access direction.

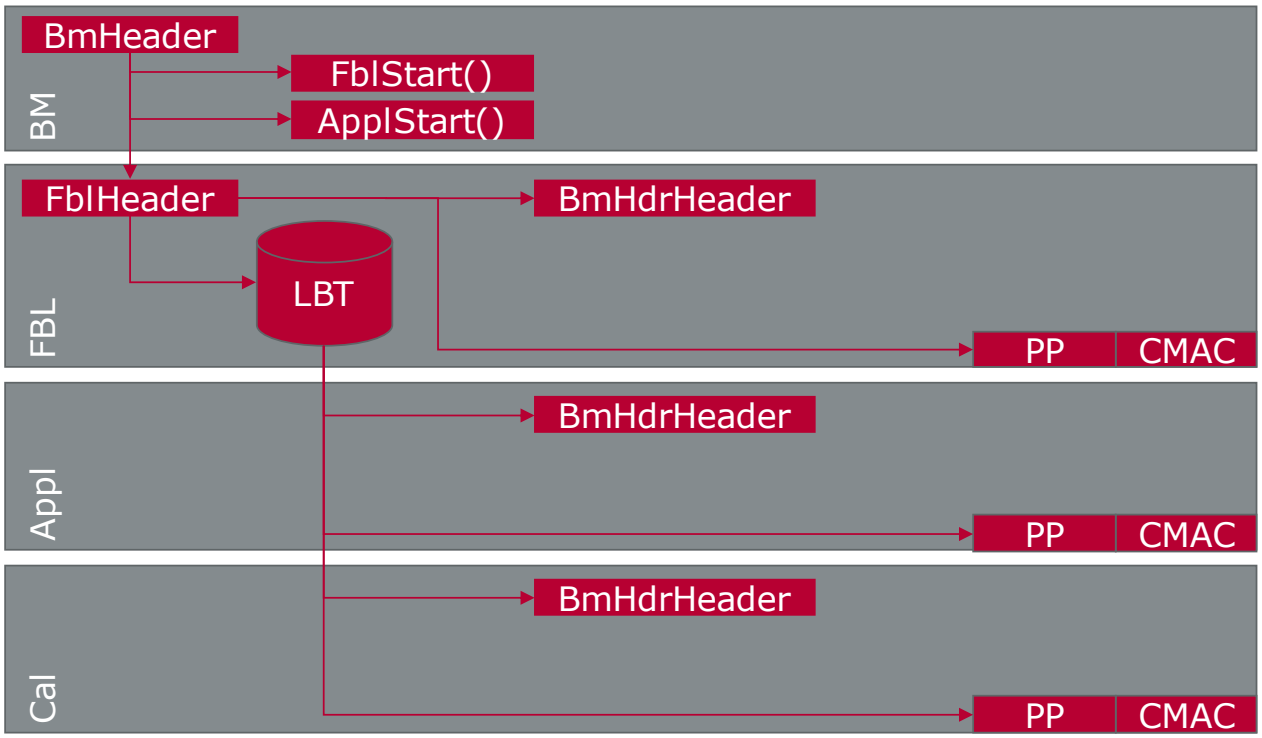


Figure 3-2 Memory Layout and Access Direction

3.3 Specific Features

The following sections describe specific features of the Boot Manager.

3.3.1 Failsafe Flash Bootloader Updater

In case the check of the bootloader fails, described in section 3.2.1 Step B, the Boot Manager enters a failsafe mechanism.

In the failsafe state the Boot Manager tries to find a valid FblBmHdrHeader in flash, which is configured to be a Flash Bootloader Updater. If a valid entry is found, the Flash Bootloader Updater is started so that the Flash Bootloader can be re-programmed. For this the following callouts are used:

- FBLBM_CALLOUT_SEARCH_FAILSAFE_UPDATER_INIT
- FBLBM_CALLOUT_SEARCH_FAILSAFE_UPDATER_HEADER_ADDRESS
- FBLBM_CALLOUT_SEARCH_FAILSAFE_UPDATER_NEXT

3.3.2 Stay In Boot (Force Boot Mode)

If the feature StayInBoot is enabled, then Step H described in section 3.2.1 is enabled. The Boot Manager checks if the StayInBoot message was already received during the configured time span by the Flash Bootloader. If the StayInBoot message check was not done yet (ApplFlag not set), the **Boot Manager** jumps into the Flash Bootloader to perform the necessary steps, otherwise continue with the next steps.

If the StayInBoot message was not received in the **Flash Bootloader**, then the Flash Bootloader jumps back into the Boot Manager to indicate that no StayInBoot message was

received. This is done by setting the ApplFlag. Otherwise it stays in the Bootloader and waits for diagnostic messages.

Figure 3-3 illustrates the sequence.

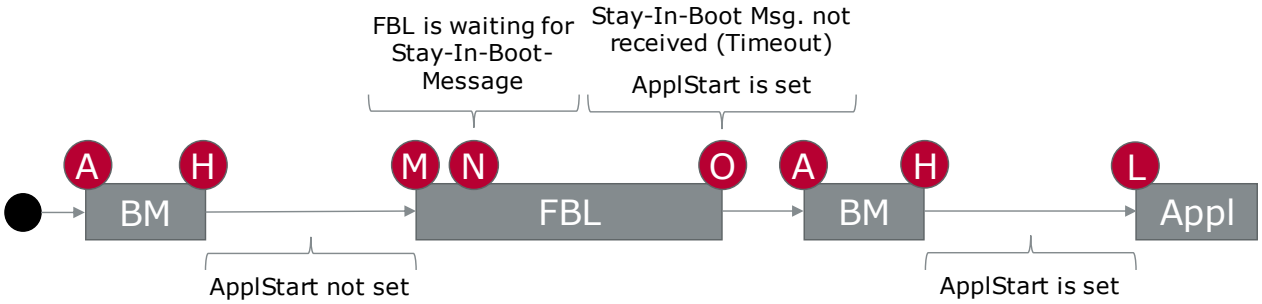


Figure 3-3 Sequence - Stay In Boot

Note
For more information about the StayInBoot feature, please refer to [3].

3.3.3 Reprogrammable Logical Block Table

If the bootloader is configured to use a reprogrammable LBT, then the Boot Manager uses it to locate the logical blocks, see Step J.

Before accessing the reprogrammable logical block table, the Boot Manager checks if the logical block containing the reprogrammable LBT is valid.

Note
For more information about the reprogrammable LBT, please refer to [3].

3.3.4 Secure Boot

During the boot sequence described in section 3.2.1, the Boot Manager performs some SecureBoot specific tasks:

> In Step E and Step F, the boot manger checks for the presence of a valid key (see section 3.3.4.3) and erased MAC (see section 3.3.4.4)

> The Boot Manager performs an authenticity check (e.g. MAC verification) over a memory area before allowing access to the memory area, e.g. over the FblBmHdrHeader. This ensures the chain of trust, as described in section 3.3.4.2.

3.3.4.1 Overview

Before starting any target (Flash Bootloader, Flash Bootloader Updater or Application) or accessing any memory outside of the Boot Manager memory area, the Boot Manager verifies the authenticity (e.g. MAC) of the corresponding area with a hardware/secret key. For example, during the logical block verification, the verification ranges defined in the FblBmHdrHeader are used.

If the authenticity is performed sequentially/synchronously, then this is referred as **Secure Boot**.

If the authenticity is performed in parallel/asynchronous/background, then this is referred as **Authentication Boot**.



Note
At the moment Authentication Boot is not supported.

3.3.4.2 Chain of Trust Concept

Chain of trust means that before accessing a memory region, the content of this region shall be verified. By doing this it is ensured that it can never happen that an unverified (untrusted) region is accessed.

The chain of trust starts with a hardware trust anchor (HTA). This can either be a hardware firmware, HSM or the Boot Manager, see

Figure 3-4. If the Boot Manger should be the HTA, then it is required to set the respective flash blocks of the Boot Manager to OTP (One-Time-Programmable) to deny any modifications to the Boot Manager (HTA).

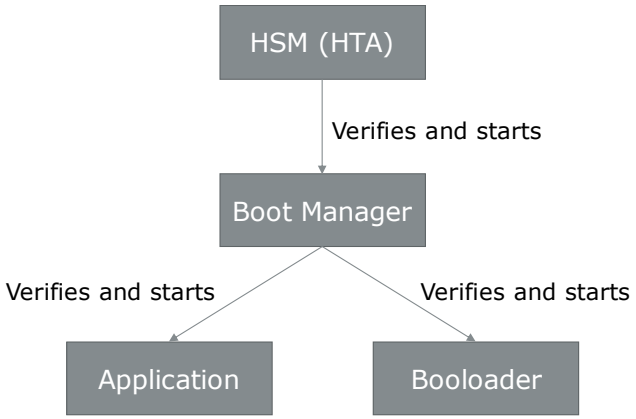


Figure 3-4 Chain of trust concept

3.3.4.3 Key Initialization

In order to use the secure boot feature, it is necessary to initialize the key values in the crypto module (SHE/HSM). There are different ways to initialize the keys:

1. Hardware supplier pre-initializes the keys
2. By downloading a special software prior to the Boot Manager and Bootloader download, which initializes the keys
3. During the first startup
4. During runtime, e.g. with a WriteDataByIdentifier service

In the following, only case 3 is described. Boot Manager has the possibility to check if the keys are not initialized. If the keys are not initialized yet, the Boot Manager can jump into the Flash Bootloader, so that it can initialize the keys into the crypto module (SHE/HSM). To enable this feature, please refer to section 5.1.1.7. The Boot Manager uses the callout FBLBM_CALLOUT_GET_KEYEMPTY to check if the keys are empty. This feature can be used to check on the production line if the keys need to be initialized and then trigger the key initialization by the Boot Loader. This sequence is illustrated in Figure 3-5.

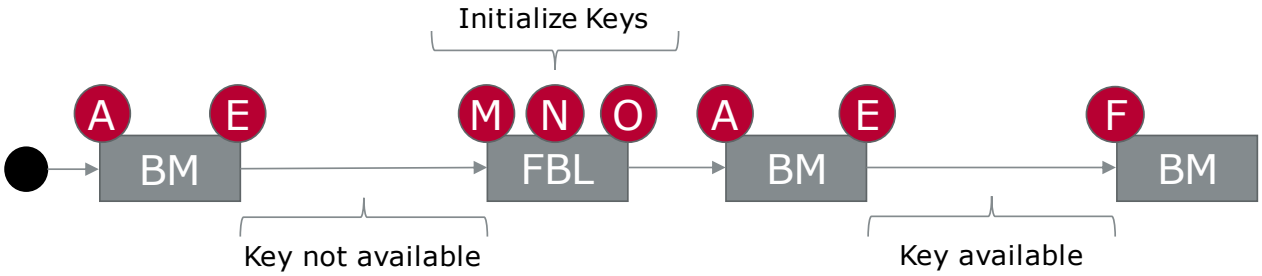


Figure 3-5 Sequence – Key Initialization

The template implementation detects the presence of a key using the MAC generation function. If the generation of a MAC is not successful then it is highly probable, that the key is not present yet.

The feature described in case 4 can also be enabled. In this case, the Bootloader does not initialize the keys during startup but waits for an input from external to start the initialization.

Key Initialization Solution	Advantages	Disadvantages
Hardware supplier (1)	The ECU supplier does not have to worry about it.	It is not known what RNG mechanism (e.g. TRNG or PRNG) is used to generate the key values.
Special software (2)	Can be performed in a safe environment. A specific or random value can be used to initialize the keys.	Additional effort to write/test the special software.
First startup (3)	No additional effort during hardware release. ECU will take care of it to initialize everything which is needed.	First startup needs to be detected only once.
Runtime (4)	Keys can be programmed after production.	Secure Boot is not active until all keys are initialized.

Table 3-1 Advantages and disadvantages of different key initialization solutions

Caution
Special care needs to be taken, if this feature is enabled, that the Boot Manager does not accidentally start the bootloader again, e.g. by an additional OTP flag. Otherwise, the keys could be altered in the field, which would jeopardize ECU security.

Note
Usually the bootloader performs a key initialization and the MAC generation in one step.

There are two different key initialization protocols, which are explained in the following sections.

3.3.4.3.1 SHE Key Update

According to the HIS specification (see [5]), when updating a key into a SHE based system, a specific protocol needs be applied (e.g. flag information how the key should be handled/used). This is also called the M1M2M3 key update protocol.

Figure 3-6 shows briefly how the M1M2M3 key update protocol looks like.

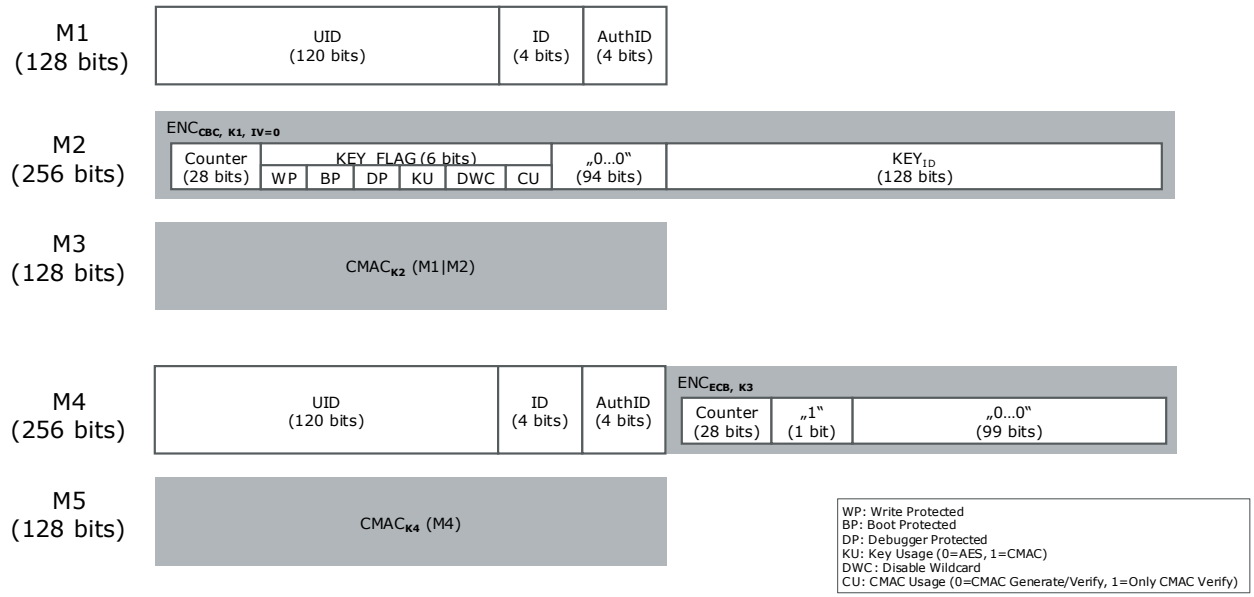


Figure 3-6 M1M2M3 key update protocol

Furthermore, to update a specific key (see ID of Figure 3-6) it is necessary to authenticate this with another key (see AuthID of Figure 3-6). The Figure 3-7 shows which key (Slot to update) can be authenticated with another key (Secret).

		non-volatile					volatile			ROM		(plaintext)
	Secret	MASTER_ECU_KEY	BOOT_MAC_KEY	BOOT_MAC	KEY_<n>	PRNG_SEED	RAM_KEY	PRNG_KEY	PRNG_STATE	SECRET_KEY	UID	
Slot to update												
MASTER_ECU_KEY		x										
BOOT_MAC_KEY		x	x									
BOOT_MAC		x	x									
KEY_<n>		x			x							
PRNG_SEED												
RAM_KEY					x					x		x
PRNG_KEY												
PRNG_STATE												
SECRET_KEY												
UID												

Figure 3-7 M1M2M3 key update policy

Caution
As non written keys have default values, which can be used to authenticate other keys, it is highly recommended to initialize all keys with a value (specific or random).

3.3.4.3.2 Plain Key Update

The keys are written into the hardware in plain format.

3.3.4.4 MAC Initialization

The Boot Manager has the possibility to check if the MAC of the Flash Bootloader is not present/not written yet. This can happen in case the ECU starts for the first time and no keys are present, see section 3.3.4.3, or the Bootloader was downloaded with a debugger during development but doesn't contain an updated MAC value.

If the MAC is not present then the Boot Manager can (optionally) jump into the Flash Bootloader, so that the Flash Bootloader can initialize its own MACs of the LBT, FblBmHdrHeader, and verification ranges. To enable this feature, please have a look at section 5.1.1.8. The Boot Manager uses the callout FBLBM_CALLOUT_GET_FBL_CMACERASED to check if the MAC is erased. This is illustrated in Figure 3-8.

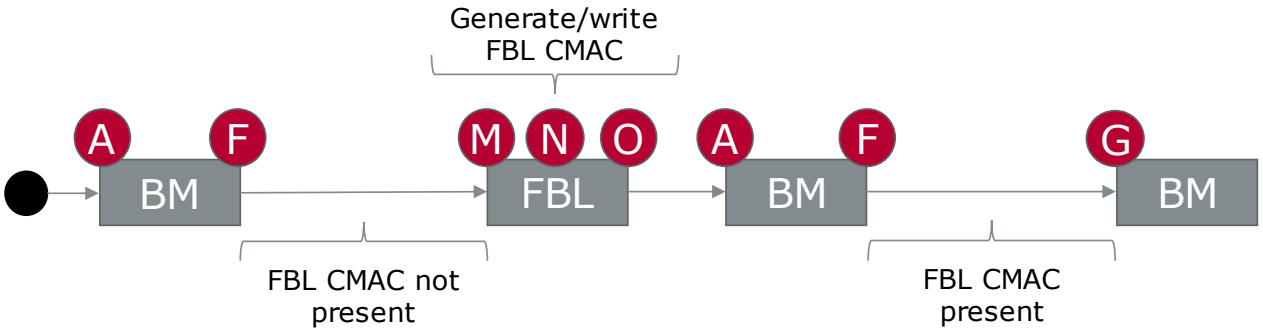


Figure 3-8 Sequence – MAC Initialization

The template implementation checks if the flash, which should contain the MAC, contains only erased value. If this is the case, it is highly probable, that the flash contains an invalid MAC value.

Caution
Special care needs to be taken, if this feature is enabled, that the Boot Manager does not accidentally start the bootloader again, e.g. by an additional OTP flag. Otherwise, the CMAC would be updated in the field, which would jeopardize the secure boot concept.

Note
This is usually a debug feature in case a new Flash bootloader image is downloaded without an updated MAC. If this feature is used in production, special care needs to be done (see caution box above).

4 Integration

Useful hints on the integration of the Boot Manager, Flash Bootloader, and Application can be found in the following sections.

4.1 Boot Manager

To integrate the Boot Manager into your project, please follow the directions described in the following sections.

4.1.1 Flash Block Table

The Boot Manager contains a flash block table (FBT). The FBT describes the flash memory regions which can be accessed by the Boot Manager. The Boot Manager should have the possibility to access the complete memory area, except its own blocks and other reserved block like the HSM. Memory regions which are currently not used should still be set to the actual memory device, as future memory layout updates (e.g. rearranging of logical blocks) are then still supported.

4.2 Flash Bootloader

The Flash Bootloader provides the FblBmHdrHeader structure for the Boot Manager. This can be achieved by adding the FblBmHdr generator to the bootloader DaVinci project and configure the module there (see section 5.1.2).

4.2.1 Presence Pattern

If the Bootloader is compiled separately (Standalone Mode), then the Bootloader requires PresencePattern for the BlockValidity. If the PresencePattern is not available, then the Bootloader is not present from the Boot Managers point of view and will not be accessed/started.

This means that during the check in Step B, the Bootloader is evaluated as invalid and therefore the Boot Manager continues with Step C.

4.2.2 Flash Block Table

In case the Flash Bootloader shall calculate the MACs on the first startup (see section 3.3.4.4), it is required to set the flash blocks for the bootloader to the actual memory device and not as “protected”.



Note
If the Boot Manager is compiled/linked with the bootloader together (see configuration switch Standalone Mode), then it is necessary to set the flash blocks of the bootloader to the actual memory device.

4.2.3 Logical Block Table

For each logical block in the LBT, the location of the FblBmHdrHeader structure must be configured (see BM Header Address configuration).

4.3 Application

The application provides the FblBmHdrHeader structure for the Boot Manager (similar to the Bootloader, see section 4.2).

4.4 Scope of Delivery

The delivery of the Boot Manager contains the files which are described in the following sections.

The dynamic files are generated by the configuration tool DaVinci Configurator 5 Pro. If the delivery is for GENy the generated files needs to be adapted manually (see chapter 5).

4.4.1 FblBm

4.4.1.1 Static Files

File Name	Description
bm_main.c	This is the source file of the Boot Manager.
bm_main.h	This is the header file of the Boot Manager.
bm_types.h	This is the common header file containing type definitions used within the Boot Manager.
bm_shared.c	This is the shared source file between the Boot Manager and the Flash Bootloader, containing the MagicFlag definition.
bm_shared.h	This is the shared header file between the Boot Manager and the Flash Bootloader, containing the MagicFlag definition.

Table 4-1 Static files of FblBm



Note
The file bm_shared.c/h needs to be compiled with the Flash Bootloader as well.

4.4.1.2 Generated Files

File Name	Description
FblBm_Cfg.h	This is the configuration header file. It contains configuration switches.
FblBm_Lcfg.c	This is the configuration source file. It contains the boot order list.

Table 4-2 Generated files of FblBm

4.4.2 FblBmHdr

4.4.2.1 Static Files

File Name	Description
bm_hdr.c	This is the source file of the Boot Manager

File Name	Description
	Header.
bm_hdr.h	This is the header file of the Boot Manager Header.
bm_hdr_types.h	This is the common header file containing type definitions used within the Boot Manager Header.

Table 4-3 Static files of FblBmHdr



Note
The static files are not required within the Flash Bootloader.

4.4.2.2 Generated Files

File Name	Description
FblBmHdr_Cfg.h	This is the configuration header file. It contains general information about the FblBmHdrHeader
FblBmHdr_Lcfg.c	This is the configuration source file. It contains the actual FblBmHdrHeader if configured and the target list.

Table 4-4 Generated files of FblBmHdr

5 Configuration

This chapter describes how the Boot Manager is configured through the configuration tool DaVinci Configurator. A list of generated files and the respective description can be found in section 4.4).

To configure the Boot Manager, a DaVinci configuration is needed. The bootloader delivery contains an example of such a configuration in folder Demo\DemoBm (usually named DemoBm.dpa) for the Boot Manager.

If the delivery supports GENy then the configuration is done manually by adaptation of template files located in BSW\FblBm_Template, BSW\FblBmHdr_Template and BSW\FblSecBoot_Template.

The following sections provide a list of all configurable Boot Manager options, together with a short description.

The following modules are relevant to the configuration of the Boot Manager:

- FblBm – Boot Manager
- FblBmHdr – Boot Manager header
- Fbl – general boot loader configuration

The following modules are relevant in case the secure boot option is used:

- FblSb – secure boot
- Csm – AUTOSAR CSM (Crypto Security Manager)
- CryIf – AUTOSAR CRYIF (Crypto Interface)
- Crypto drivers of the hardware (depending on the use case):
  - 3rd Party HTA:
    - AUTOSAR CRYPTO – CRYPTO CRYWRAPPER (if required)
    - 3rd Party CRYPTO Driver
  - Vector HTA:
    - AUTOSAR CRYPTO – CRYPTO CRYWRAPPER
    - Vector CRYPTO Driver
  - Vector vHSM
    - vHSM Driver

The following sections describes the DaVinci modules and the configuration aspects.

Note
There is no generator for GENy available. In this case the “generated files” described in section 4.4 need to be adapted manually. Template files are provided in BSW\<module>_Template.

5.1 FblBm

The following sections describe the configuration of the FblBm component.

5.1.1 FblBm | FblBmGeneral

The general Boot Manager configuration is located in the module **FblBm | FblBmGeneral**.

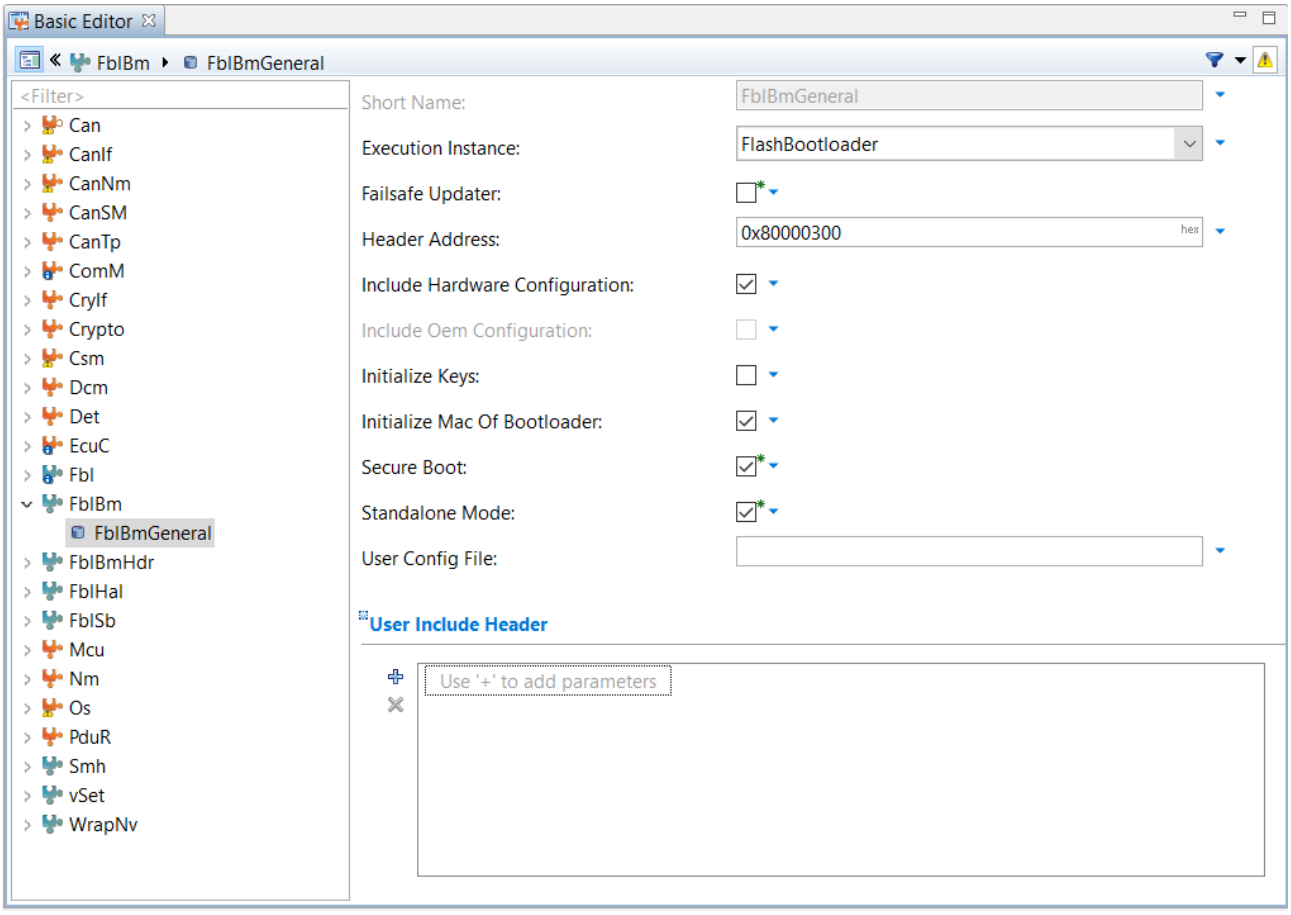


Figure 5-1 DaVinci Configurator: FblBmGeneral Configuration

5.1.1.1 Execution Instance

This parameter specifies the context in which the software is built.

The value “Others” is a placeholder in case this component is required in other builds as well, e.g. application.

5.1.1.2 Failsafe Updater

By enabling this option, the Boot Manager will check on startup if a valid Flash Bootloader Updater is present. This is done in case the Flash Bootloader is corrupted or erased due to an interrupted Flash Bootloader Updater sequence. For more information, please refer to section 3.3.1.



Caution
This feature needs to be enabled, if the Flash Bootloader shall be updatable in the field.

5.1.1.3 Header Address

This is the header address of the Boot Manager header structure `FblBmHeader`.



Caution
Make sure to link the `FblBmHeader` structure to the same address as in the configuration. See [1] on how to map the `FblHeader` structure by means of the `FBLBM_HEADER_START_SEC_CONST` and `FBLBM_HEADER_STOP_SEC_CONST` macro.

5.1.1.4 Include Generic Configuration



Changes
The generic configuration is included automatically if an execution instance is selected which contains the Bootmanager.

5.1.1.5 Include Hardware Configuration

This parameter defines whether the hardware configuration files bm_hw_ap.h and bm_hw_cfg.h will be included by the Bootmanager.

The “bm_hw_cfg.h” file can be used to overwrite the Boot Manager callout functions with hardware specific implementations. See section 6.1 for details on how to overwrite the Boot Manager API callout functions.

5.1.1.6 Include Oem Configuration

This parameter defines whether the OEM configuration file bm_oem_ap.h and bm_oem_cfg.h will be included by the Bootmanager.

The “bm_oem_cfg.h” file can be used to overwrite the Boot Manager callout functions with OEM specific implementations. See section 6.1 for details on how to overwrite the Boot Manager API callout functions.

5.1.1.7 Initialize Keys

By enabling this option, the Boot Manager will check if the necessary keys for the secure boot validation are already initialized. For more information, please refer to section 3.3.4.3.

5.1.1.8 Initialize Mac Of Bootloader

By enabling this option, the Boot Manager will check if the CMAC of the Flash Bootloader image for the secure boot validation is present. For more information, please refer to section 3.3.4.4.

5.1.1.9 Secure Boot

Activation of Flash Bootloader, Flash Bootloader Updater, and application integrity and authenticity check by using a hardware trust anchor (HTA), see section 3.3.4.2.

See section 3.2 for details about the secure boot sequence and section 5.2.3.4 for the configuration of secure boot.

Note
Secure boot uses a HTA. To access the HTA, the Boot Manager uses the AUTOSAR CSM API, which requires the complete crypto stack.

5.1.1.10 Standalone Mode

By activating this option, the Boot Manager is an independent project and needs to be compiled/linked separately. Standalone Mode needs to be enabled for secure boot and/or for a reset-safe Flash Bootloader Updater option (see [4] for more information). The Boot Manager must be placed in separate flash blocks than the Bootloader. This allows that the Bootloader gets erased by a Bootloader Updater without affecting the Boot Manager.

If Standalone Mode is disabled, the Boot Manager needs to be built as part of the Flash Bootloader project and does not need to be placed in separate flash blocks.

5.1.1.11 User Config File

Reference to an external text file that will be included during generation to “FblBm_Cfg.h”.

The content of the user configuration file will be added at the end of the generated module configuration file and allows modification or extending the generated code.

5.1.2 FblBm | FblBmBootOrder

This container configures the order in which boot targets are checked and started.

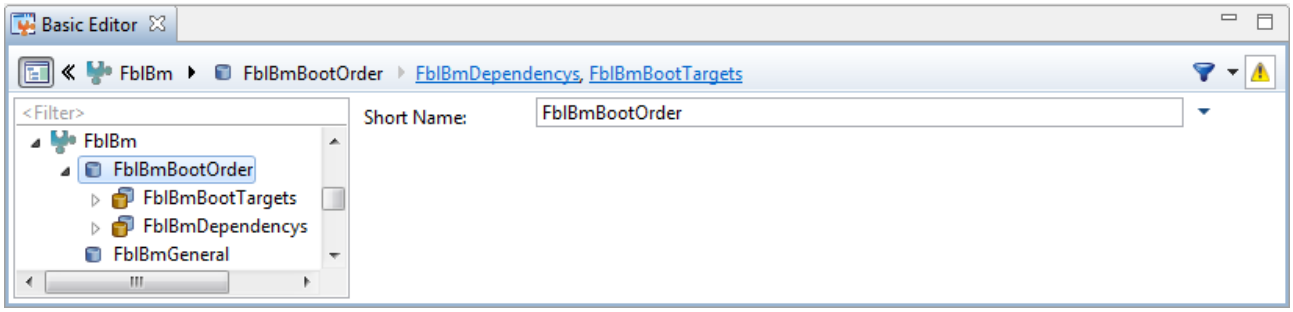


Figure 5-2 DaVinci Configurator: FblBmBootOrder

Note
This container is optional and not present per default. It can be added by right-clicking on the FblBm module, Create Sub Container and selecting FblBmBootOrder.

Note
The container is only necessary for the Boot Manager configuration. If the container is not present, then the Boot Manager will always and only start the Bootloader.

5.1.2.1 FblBm | FblBmBootOrder | FblBmBootTargets

This container contains a list of boot targets.

Please note, that the Bootloader is separately started (see Figure 3-1) and therefore not required to be added explicitly in the BootTarget list.

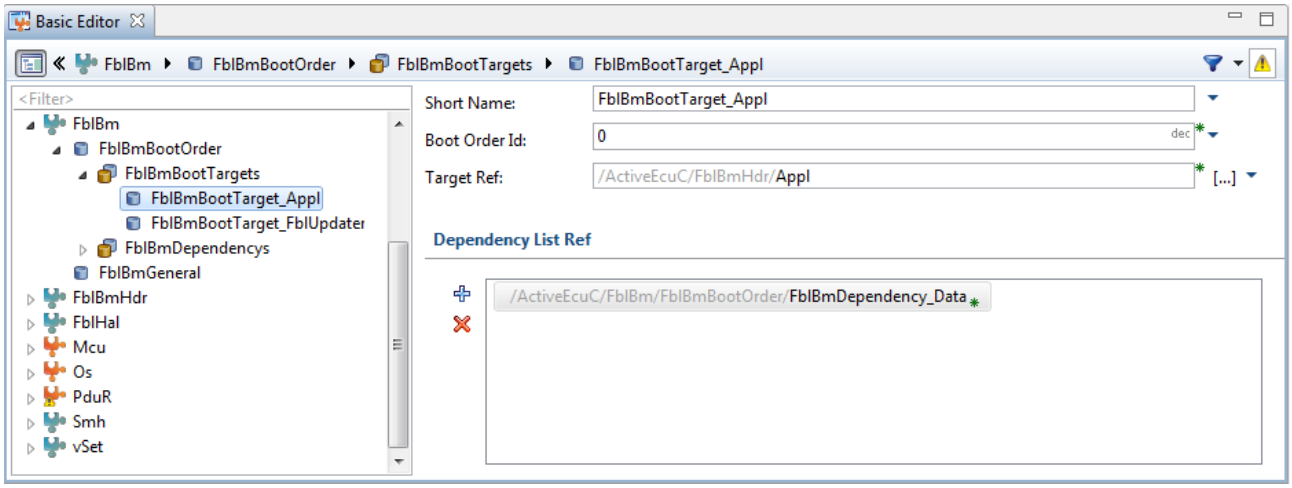


Figure 5-3 DaVinci Configurator: FblBmBootTargets

5.1.2.1.1 Boot Order Id

With the boot order ID it is possible to specify the order in which the Boot Manager checks and starts the boot targets.

5.1.2.1.2 Target Ref

This parameter references to a FblBmHdrTarget, which shall be used as a boot target.

5.1.2.1.3 Dependency List Ref

A list of FblBmDependencyTargets, which are checked by the Boot Manager additionally to the actual Boot Target.

5.1.2.2 FblBm | FblBmBootOrder | FblBmDependencyTargets

This container contains a list of dependency targets, which can be added to the Dependency List Ref of a Boot Target.

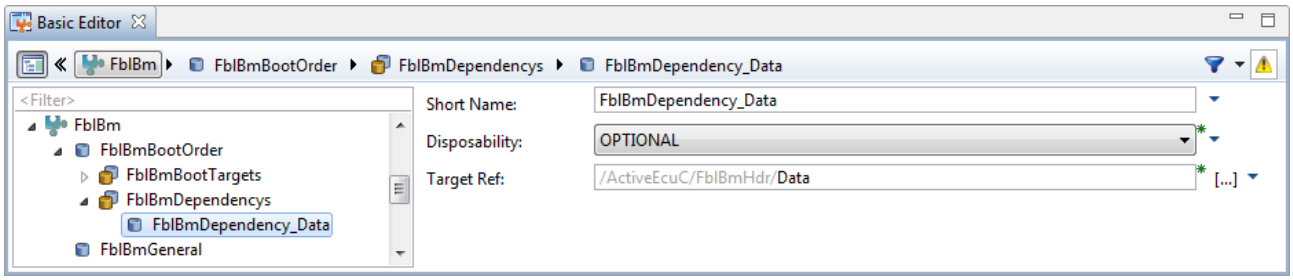


Figure 5-4 DaVinci Configurator: FblBmDependencyTargets

5.1.2.2.1 Disposablity

Specify if the dependency target is mandatory to start the boot target or optional.

Note
In case Secure Boot is enabled, then the CMAC needs to be valid and is not dependent on the disposability.

5.1.2.2.2 Target Ref

This parameter references to a FblBmHdrTarget, which shall be used as a Dependency Target.

5.2 FblBmHdr

The following sections describe the configuration of the FblBmHdr component.

5.2.1 FblBmHdr | FblBmHdrGeneral

The Boot Manager header table contains the necessary meta information for integrity and authenticity checking and starting the according boot target.

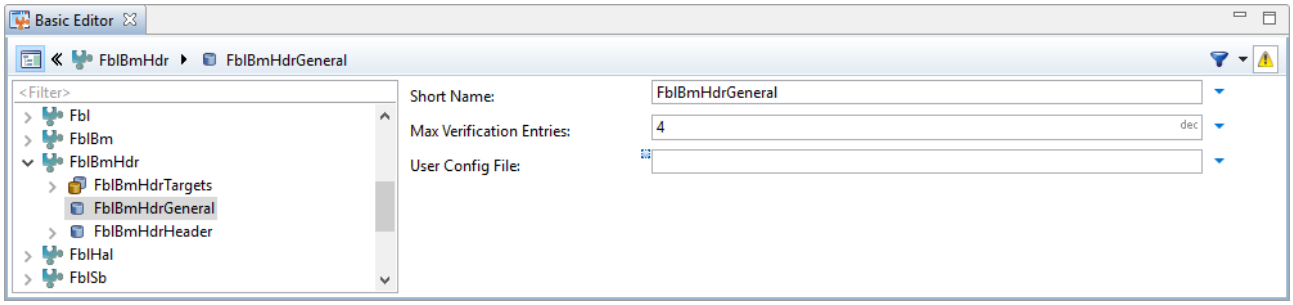


Figure 5-5 DaVinci Configurator: FblBmHdrGeneral Configuration

5.2.1.1 Max Verification Entries

If secure boot is enabled, this attribute defines the maximum number of verification areas per verification target (see section 5.2.3.4 for the configuration of the verification areas). Otherwise, this attribute is not used.

Caution
Max Verifications Entries value shall be the same for all the projects (Boot Manager, Bootloader, …).

5.2.1.2 User Config File

Reference to an external text file that will be included during generation to “FblBmHdr_Cfg.h”.

The content of the user configuration file will be added at the end of the generated module configuration file and allows modification or extending the generated code.

5.2.2 FblBmHdr | FblBmHdrTargets

The FblBmHdrTargets list is used to define the logical blocks that are relevant for starting a specific Boot Target and corresponding Dependency Targets.

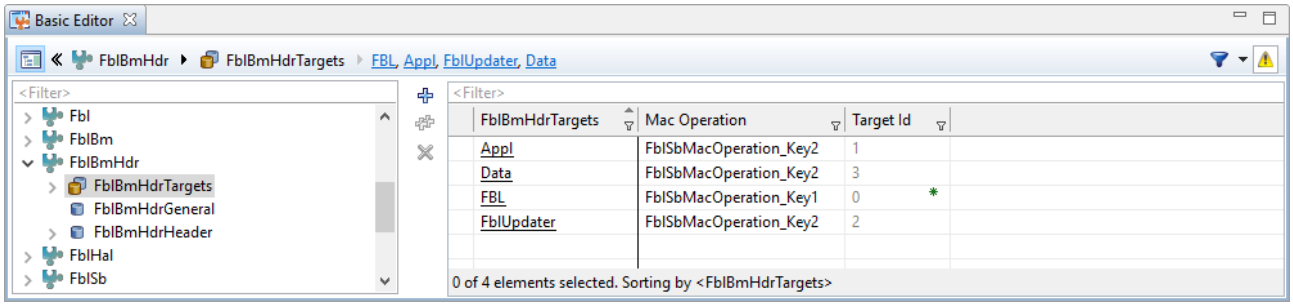


Figure 5-6 DaVinci Configurator: FblBmHdrTargets Configuration

5.2.2.1 Mac Operation

In case of secure boot, this entry refers to a FblSbMacOperations (see configuration in [6]). If secure boot is not used, this entry must be left empty.

5.2.3 FblBmHdr | FblBmHdrHeader

The FblBmHdrHeader table needs to be configured and linked for every target (e.g. for the Flash Bootloader). Figure 5-7 shows an example configuration for the Flash Bootloader.

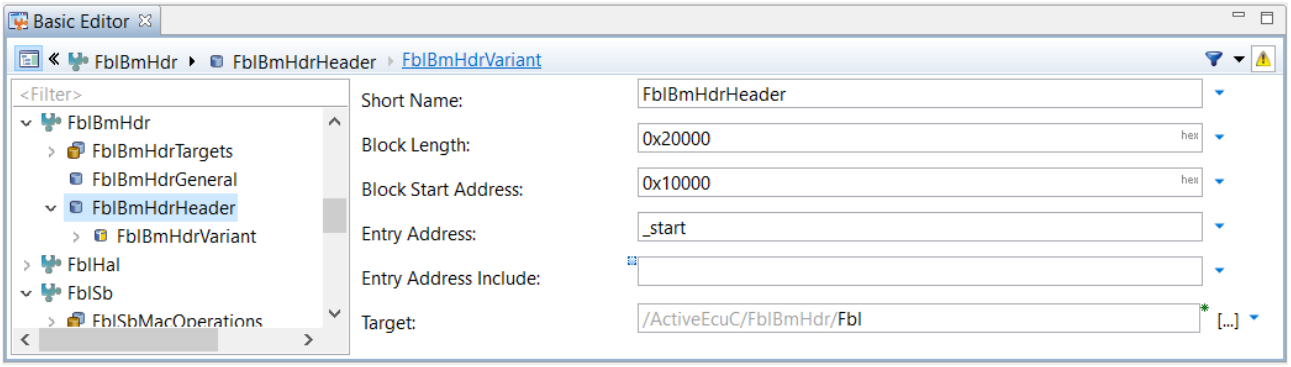


Figure 5-7 DaVinci Configurator: FblBmHdrHeader Configuration



Note
This container is optional and not present per default. It can be added by right-clicking on the FblBmHdr module, Create Sub Container, and select FblBmHdrHeader.

5.2.3.1 Block Length

This attribute is used to define the length of the corresponding target area.

5.2.3.2 Block Start Address

This attribute is used to define the start address of the corresponding target area.

5.2.3.3 Entry Address

Entry point of the corresponding target. It can be specified either as a physical address value, or as a valid linker symbol.

If the Entry Address is specified as a physical address, DaVinci makes sure that the value is within the target's address area, which is defined by the Block Start Address and Block Length. If the Entry Address is defined as a linker symbol, a runtime check makes sure that the configuration is correct.



Note
If the target is not a boot target but instead used as a dependency target (e.g. a calibration target), the entry address can be left empty.

5.2.3.4 Entry Address Include

Specify an include file, which contains the declaration of the FblBmHdrEntryAddress symbol in case a linker symbol is used as Entry Address.

If no include file is specified and a linker symbol is used, an extern uint8 array is declared to resolve potential compiler errors.

5.2.3.5 Target

This attribute is used to specify the target handle for which this header should be generated.

5.2.3.6 FblBmHdr | FblBmHdrHeader | FblBmHdrVariant

Here it is possible to select between two different header variants. The two variants are described in the next chapters.

The Boot Manager supports only one variant. This means that it is not possible for the bootmanager to decide during runtime which variant should be used.

If available, the SecureBoot module (FblSb) makes sure that the correct variant for this SecureBoot library is used.

5.2.3.6.1 FblBmHdr | FblBmHdrHeader | FblBmHdrVariant | FblBmHdrVerificationList | FblBmFblBmHdrVerificationRanges

The FblBmHdrVerificationRanges table is used to define the memory areas that shall be considered by the Boot Manager to perform the verification checks for secure boot. The

complete memory must be written to a definite value. Memory in erased or undefined state is not allowed.

Figure 5-8 shows an example configuration for the Flash Bootloader.

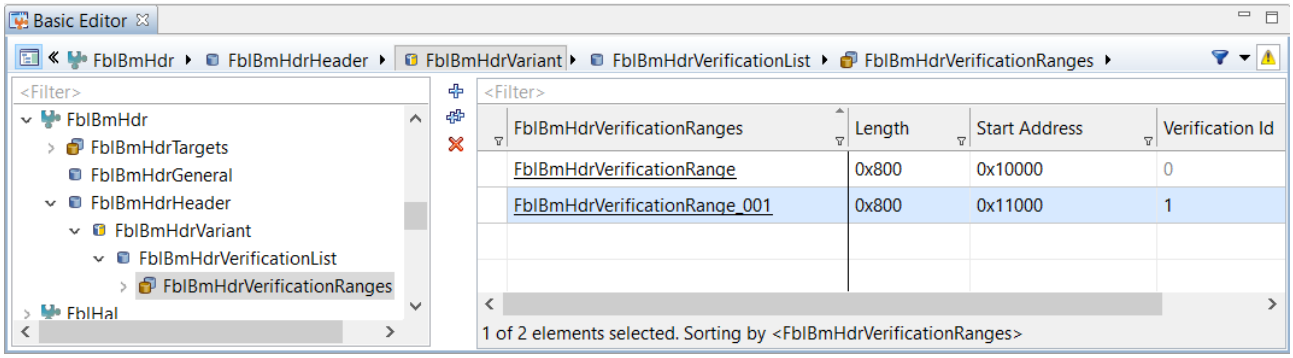


Figure 5-8 DaVinci Configurator: FblBmHdrVerificationRanges Configuration

Note
The Boot Manager checks if the provided Entry Address is within one of the verification ranges.

Caution
For the verification area of the Flash Bootloader, the memory area configuration needs to be consistent with the configuration of Block Start Address and Block Length.

5.2.3.6.1.1 Length

Length of the secure boot verification area.

5.2.3.6.1.2 Start Address

Start address of the secure boot verification area.

5.2.3.6.2 FblBmHdr | FblBmHdrHeader | FblBmHdrVariant | FblBmHdrAuthenticationHeader

The authentication header is used to specify a memory location where a specific header for secure boot is located. The header can contain a segment list, which is used by the corresponding SecureBoot library to update MACs internally (e.g. within the HSM).

Figure 5-9 shows an example configuration for the Bootloader.

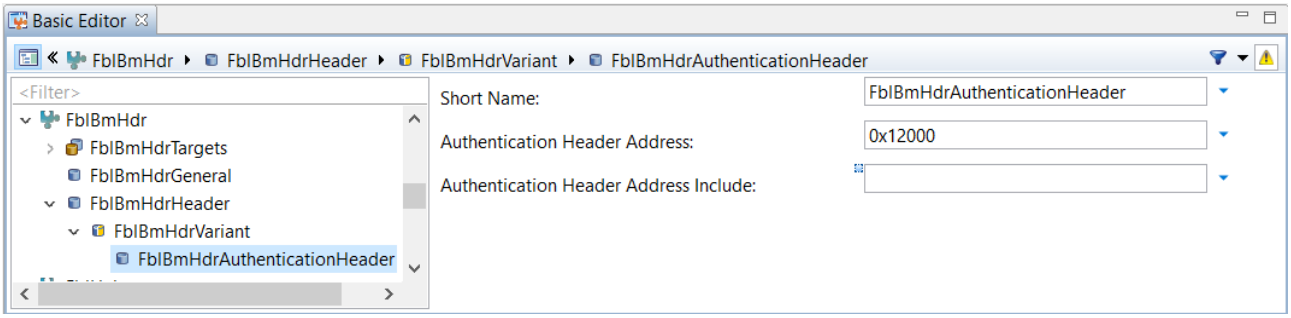


Figure 5-9 DaVinci Configurator: FblBmHdrAuthenticationHeader Configuration

5.2.3.6.2.1 Authentication Header Address

Authentication header address of the corresponding target. It can be specified either as a physical address value, or as a valid linker symbol.

If the Authentication Header Address is specified as a physical address, DaVinci makes sure that the value is within the target's address area, which is defined by the Block Start Address and Block Length.

5.2.3.6.2.2 Authentication Header Address Include

Specify an include file, which contains the declaration of the FblBmHdrAuthenticationHeaderAddress symbol in case a linker symbol is used as Authentication Header Address.

If no include file is specified and a linker symbol is used, an extern uint8 array is declared to resolve potential compiler errors.

5.3 Fbl

The following sections describe the configuration of the Fbl component (Boot Manager relevant options).

5.3.1 Fbl | FblGeneral

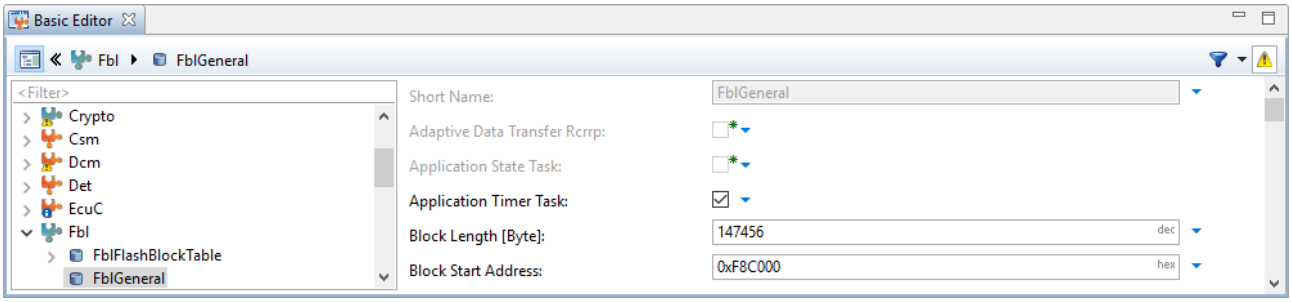


Figure 5-10 DaVinci Configurator: FblGeneral Configuration

5.3.1.1 Block Length

Length of the Flash Bootloader area in bytes. This parameter needs to be aligned with the definition of the corresponding Block Length in the FblBmHdrHeader configuration.

5.3.1.2 Block Start Address

Start address of the Flash Bootloader area. This parameter needs to be aligned with the definition of the corresponding Start Address in the FblBmHdrHeader configuration.

5.3.1.3 Fbl | FblLogicalBlockTable | FblLogicalBlocks

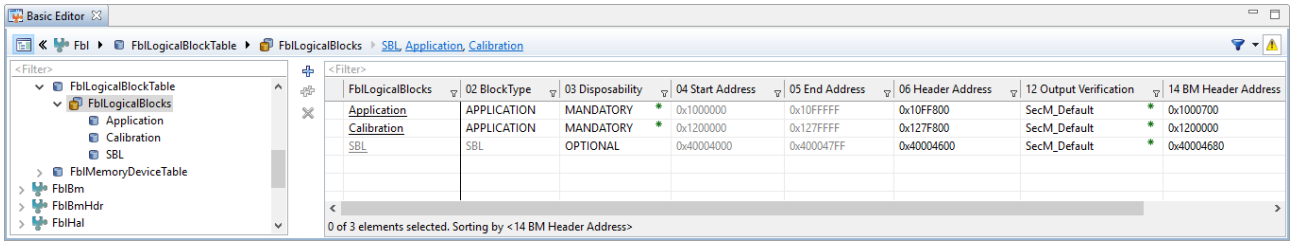


Figure 5-11 DaVinci Configurator: FblLogicalBlocks Configuration

5.3.1.3.1 BM Header Address

This attribute defines the base address of the corresponding FblBmHdrHeader. The value has to be within the start and end address of the logical block.



Note
Make sure that the FblBmHdrHeader is linked at the configured BM Header Address in the corresponding logical block.

5.4 Project Settings

There are further project specific setting depending on the used crypto driver/module.

5.4.1 Vector vHSM

In case the Vector vHSM solution is used, the vHSM Pre-Config file needs to be referenced by the host configuration (e.g. Boot Manager, Flash Bootloader). After the reference has been added the DaVinci Configurator needs to be restarted to apply the changes.

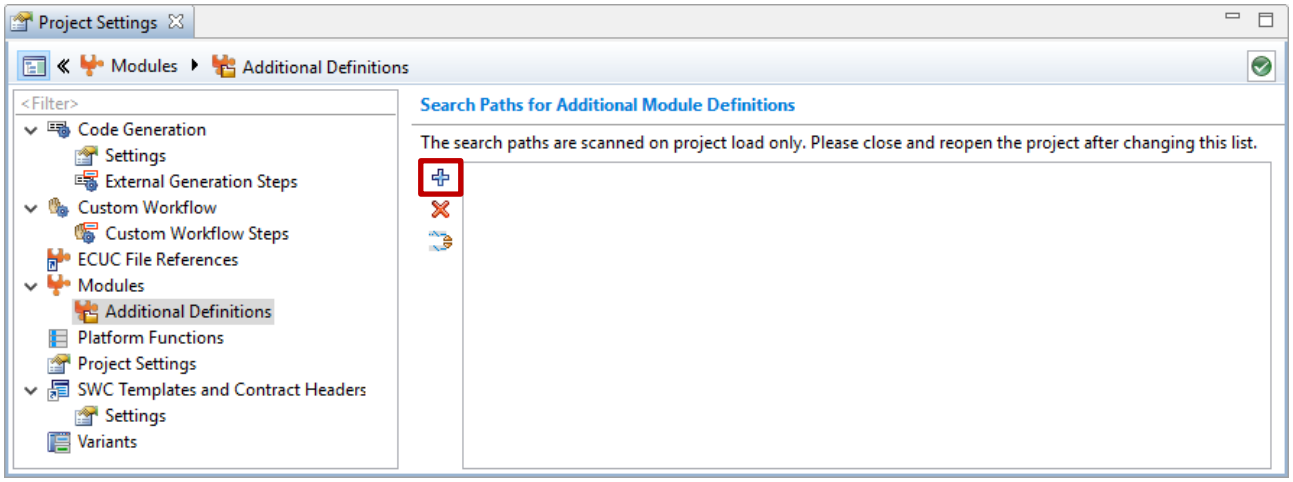


Figure 5-12 DaVinci Configurator: Add additional Module Definitions

The location of the Pre-Config file can be configured within the vHSM configuration.

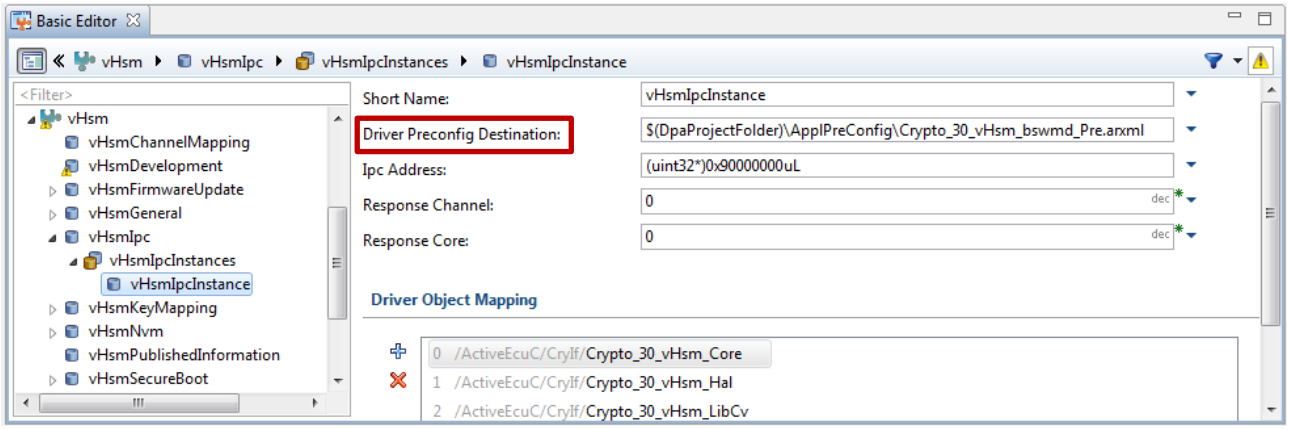


Figure 5-13 DaVinci Configurator: Configure destination of the Pre-Config file

6 API Description

The API of the Boot Manger is described in this chapter.

6.1 Overwriting API Callout Functions

The Boot Manager comes with a set of configurable hook/callout functions that can be used to adapt the Boot Manager for project/hardware specific requirements. To encapsulate the hook/callout function names, the function names are mapped by a set of macros in the template file “bm_ap_cfg.h”.

6.1.1 Overview of Callout Functions (FBLBM_CALLOUT_)

The following table provides an overview of the callout macros and a short description.

Overwrite Hook	Description
FBLBM_CALLOUT_POWER_ON_PRE_HW	Power on hardware initialization pre-hook
FBLBM_CALLOUT_POWER_ON_PRE_OEM	Power on OEM-specific initialization pre-hook
FBLBM_CALLOUT_POWER_ON_PRE	General power on pre-hook
FBLBM_CALLOUT_POWER_ON_POST_HW	Power on hardware initialization post-hook
FBLBM_CALLOUT_POWER_ON_POST_OEM	Power on OEM-specific initialization post-hook
FBLBM_CALLOUT_POWER_ON_POST	General power on post-hook
FBLBM_CALLOUT_INIT_PRE_HW	Hardware initialization pre-hook
FBLBM_CALLOUT_INIT_PRE_OEM	OEM-specific initialization pre-hook
FBLBM_CALLOUT_INIT_PRE	General initialization pre-hook
FBLBM_CALLOUT_INIT_POST_HW	Hardware initialization post-hook
FBLBM_CALLOUT_INIT_POST_OEM	OEM-specific initialization post-hook
FBLBM_CALLOUT_INIT_POST	General initialization post-hook
FBLBM_CALLOUT_DEINIT_PRE_HW	Hardware deinitialization pre-hook
FBLBM_CALLOUT_DEINIT_PRE_OEM	OEM-specific deinitialization pre-hook
FBLBM_CALLOUT_DEINIT_PRE	General deinitialization pre-hook
FBLBM_CALLOUT_DEINIT_POST_HW	Hardware deinitialization post-hook
FBLBM_CALLOUT_DEINIT_POST	OEM-specific de initialization post-hook
FBLBM_CALLOUT_DEINIT_POST_OEM	General initialization post-hook
FBLBM_CALLOUT_RESET	Issue hardware reset
FBLBM_CALLOUT_CHECK_TARGET_VALIDITY	Check overall validity of boot target
FBLBM_CALLOUT_IS_VALIDBLOCK	Check logical block validity of boot target
FBLBM_CALLOUT_CALL_TARGET	Start boot target
FBLBM_CALLOUT_CALL_FBL	Start Flash Bootloader target
FBLBM_CALLOUT_CALL_FBL_INIT_KEYS	Start Flash Bootloader to initialize AES keys
FBLBM_CALLOUT_CALL_FBL_INIT_CMAC	Start Flash Bootloader to initialize bootloader CMAC
FBLBM_CALLOUT_CALL_FAILSAFE_UPDATER	Start Flash Bootloader Updater target
FBLBM_CALLOUT_GET_KEYEMPTY	Check if AES keys are already initialized

Overwrite Hook	Description
FBLBM_CALLOUT_GET_FBL_CMACERASED	Check if bootloader CMAC was already initialized
FBLBM_CALLOUT_SEARCH_FAILSAFE_UPDATER_INIT	Initialize search for Flash Bootloader Updater FblBmHdrHeader
FBLBM_CALLOUT_SEARCH_FAILSAFE_UPDATER_HEADER_ADDRESS	Flash bootloader updater FblBmHdrHeader search iteration function
FBLBM_CALLOUT_SEARCH_FAILSAFE_UPDATER_NEXT	Proceed to next bootloader updater FblBmHdrHeader location
FBLBM_CALLOUT_FATAL_ERROR	Boot manager error hook

Table 6-1 Callout Functions Overview



Note
The following sections describe the API of the callout macros. As described above, these are usually only macros, which can be defined to real functions (e.g. in the file bm_ap_cfg.h).

6.1.1.1 RESET

Prototype

```
void FBLBM_CALLOUT_RESET
(
   void
)
```

Parameter	
-	-

Return code	
-	-

Functional Description

This function is responsible for resetting the ECU. For example, you may execute a restart instruction (if available), jump to the reset vector, or use the watchdog. The choice is up to you and depends on your ECU hardware. This function is called when it is necessary to reset the ECU.

Particularities and Limitations

> Jumping directly to the ECU's reset vector should be considered a last-choice option. Some ECUs contain registers that may be accessed only once – it is possible that the FBL will be unable to re-initialize them when it is restarted via a jump to the reset vector.
> Do not waste too much time in this function (e.g. through an endless loop waiting for a long-lasting watchdog trigger). This can cause unwanted delay on the network operation or service tester.

Call context

-

Table 6-2 API Description: FBLBM_CALLOUT_RESET

6.1.1.2 FBLBM_CALLOUT_POWER_ON_PRE_HW

Prototype	
`void FBLBM_CALLOUT_POWER_ON_PRE_HW ( void )`	
Parameter	
-	-
Return code	
-	-
Functional Description	
Project/ECU specific hardware power-on initialization. This function is called right after the startup initialization.	
Particularities and Limitations	
-	
Call context	
-	

Table 6-3 API Description: FBLBM_CALLOUT_POWER_ON_PRE_HW

6.1.1.3 POWER_ON_PRE_OEM

Prototype	
`void FBLBM_CALLOUT_POWER_ON_PRE_OEM ( void )`	
Parameter	
-	-
Return code	
-	-
Functional Description	
OEM specific hardware power-on initialization. This function is called after the hardware specific power-on initialization.	
Particularities and Limitations	
-	
Call context	
-	

Table 6-4 API Description: FBLBM_CALLOUT_POWER_ON_PRE_OEM

6.1.1.4 POWER_ON_PRE

Prototype	
`void FBLBM_CALLOUT_POWER_ON_PRE ( void )`	
Parameter	
-	-
Return code	
-	-
Functional Description	
General project/ECU specific power-on initialization. This function is called after the OEM specific power-on initialization.	
Particularities and Limitations	
-	
Call context	
-	

Table 6-5 API Description: FBLBM_CALLOUT_POWER_ON_PRE

6.1.1.5 POWER_ON_POST_HW

Prototype	
`void FBLBM_CALLOUT_POWER_ON_POST_HW ( void )`	
Parameter	
-	-
Return code	
-	-
Functional Description	
Project/ECU specific hardware power-on software initialization post hook. This API is called after the initialization of the memory driver and before initialization of the watchdog.	
Particularities and Limitations	
-	
Call context	
-	

Table 6-6 API Description: FBLBM_CALLOUT_POWER_ON_POST_HW

6.1.1.6 POWER_ON_POST_OEM

Prototype	
`void FBLBM_CALLOUT_POWER_ON_POST_OEM ( void )`	
Parameter	
-	-
Return code	
-	-
Functional Description	
OEM specific power-on initialization post hook. This API is called after the hardware specific power on initialization post hook.	
Particularities and Limitations	
-	
Call context	
-	

Table 6-7 API Description: FBLBM_CALLOUT_POWER_ON_POST_OEM

6.1.1.7 POWER_ON_POST

Prototype	
`void FBLBM_CALLOUT_POWER_ON_POST ( void )`	
Parameter	
-	-
Return code	
-	-
Functional Description	
Project/ECU specific power-on initialization post hook. This API is called after the OEM specific power on initialization post hook.	
Particularities and Limitations	
-	
Call context	
-	

Table 6-8 API Description: FBLBM_CALLOUT_POWER_ON_POST

6.1.1.8 INIT_PRE_HW

Prototype	
`void FBLBM_CALLOUT_INIT_PRE_HW ( void )`	
Parameter	
-	
Return code	
-	-
Functional Description	
Hardware specific software initialization. This API is called after the power on initialization.	
Particularities and Limitations	
-	
Call context	
-	

Table 6-9 API Description: FBLBM_CALLOUT_INIT_PRE_HW

6.1.1.9 INIT_PRE_OEM

Prototype	
`void FBLBM_CALLOUT_INIT_PRE_OEM ( void )`	
Parameter	
-	-
Return code	
-	-
Functional Description	
OEM specific software initialization. This API is called after the hardware specific software initialization.	
Particularities and Limitations	
-	
Call context	
-	

Table 6-10 API Description: FBLBM_CALLOUT_INIT_PRE_OEM

6.1.1.10 INIT_PRE

Prototype	
`void FBLBM_CALLOUT_INIT_PRE ( void )`	
Parameter	
-	-
Return code	
-	-
Functional Description	
Project/ECU specific software initialization. This API is called after the OEM specific software initialization.	
Particularities and Limitations	
-	
Call context	
-	

Table 6-11 API Description: FBLBM_CALLOUT_INIT_PRE

6.1.1.11 DEINIT_PRE_HW

Prototype	
`void FBLBM_CALLOUT_DEINIT_PRE_HW ( void )`	
Parameter	
-	-
Return code	
-	-
Functional Description	
Project/ECU specific hardware deinitialization hook. This API is called before starting a boot target and **before** the watchdog handling is disabled in the Boot Manager. It can be used to implement a hardware specific shutdown routine.	
Particularities and Limitations	
-	
Call context	
-	

Table 6-12 API Description: FBLBM_CALLOUT_DEINIT_PRE_HW

6.1.1.12 DEINIT_PRE_OEM

Prototype	
`void FBLBM_CALLOUT_DEINIT_PRE_OEM ( void )`	
Parameter	
-	-
Return code	
-	-
Functional Description	
Project/ECU specific OEM deinitialization hook. This API is called after the hardware specific deinitialization hook.	
Particularities and Limitations	
-	
Call context	
-	

Table 6-13 API Description: FBLBM_CALLOUT_DEINIT_PRE_OEM

6.1.1.13 DEINIT_PRE

Prototype	
`void FBLBM_CALLOUT_DEINIT_PRE ( void )`	
Parameter	
-	-
Return code	
-	-
Functional Description	
Project/ECU specific general deinitialization hook. This API is called after the OEM specific deinitialization hook.	
Particularities and Limitations	
-	
Call context	
-	

Table 6-14 API Description: FBLBM_CALLOUT_DEINIT_PRE

6.1.1.14 DEINIT_POST_HW

Prototype	
`void FBLBM_CALLOUT_DEINIT_POST_HW ( void )`	
Parameter	
-	-
Return code	
-	-
Functional Description	
Project/ECU specific hardware deinitialization post hook. This API is called before starting a boot target and **after** the watchdog handling is disabled in the Boot Manager. It can be used to implement a hardware specific shutdown routine.	
Particularities and Limitations	
-	
Call context	
-	

Table 6-15 API Description: FBLBM_CALLOUT_DEINIT_POST_HW

6.1.1.15 DEINIT_POST_OEM

Prototype	
`void FBLBM_CALLOUT_DEINIT_POST_OEM ( void )`	
Parameter	
-	-
Return code	
-	-
Functional Description	
Project/ECU specific OEM deinitialization post hook. This API is called after the hardware specific deinitialization post hook.	
Particularities and Limitations	
-	
Call context	
-	

Table 6-16 API Description: FBLBM_CALLOUT_DEINIT_POST_OEM

6.1.1.16 DEINIT_POST

Prototype	
`void FBLBM_CALLOUT_DEINIT_POST ( void )`	
Parameter	
-	-
Return code	
-	-
Functional Description	
Project/ECU specific general deinitialization post hook. This API is called after the OEM specific deinitialization post hook.	
Particularities and Limitations	
-	
Call context	
-	

Table 6-17 API Description: FBLBM_CALLOUT_DEINIT_POST

6.1.1.17 CHECK_TARGET_VALIDITY

Prototype	
`tFblResult FBLBM_CALLOUT_CHECK_TARGET_VALIDITY ( tFblBmHdrTargetHandle targetHandle )`	
Parameter	
`targetHandle [in]`	Target handle (e.g. Flash Bootloader, application, Flash Bootloader Updater)
Return code	
`kFblOk`	Target valid
`kFblFailed`	Target invalid
Functional Description	
Check the validity of the boot target (e.g. presence pattern or application overall valid flag).	
Particularities and Limitations	
-	
Call context	
-	

Table 6-18 API Description: FBLBM_CALLOUT_CHECK_TARGET_VALIDITY

6.1.1.18 IS_VALIDBLOCK

Prototype	
`tFblResult FBLBM_CALLOUT_IS_VALIDBLOCK ( tFblBmHdrTargetHandle targetHandle, tBlockDescriptor *pLogicalBlock )`	
Parameter	
`targetHandle [in]`	Target handle (e.g. Flash Bootloader, application, Flash Bootloader Updater)
`pLogicalBlock [in]`	Pointer to current logical block
Return code	
`kFblOk`	Logical block valid
`kFblFailed`	Logical block invalid
Functional Description	
Check the validity of a specific logical block.	
Particularities and Limitations	
-	
Call context	
-	

Table 6-19 API Description: FBLBM_CALLOUT_IS_VALIDBLOCK

6.1.1.19 CALL_FBL

Prototype	
`void FBLBM_CALLOUT_CALL_FBL ( const V_MEMRAM1 tFblBmHdrHeader V_MEMRAM2 V_MEMRAM3 * bmHeader )`	
Parameter	
`bmHeader [in]`	FblBmHdrHeader of the software which shall be started.
Return code	
–	-
Functional Description	
Start Flash Bootloader target with the specified entry address. This function will never return because control is handed over to the called software instance.	
Particularities and Limitations	
-	
Call context	
-	

Table 6-20 API Description: FBLBM_CALLOUT_CALL_FBL

6.1.1.20 CALL_TARGET

Prototype	
`void FBLBM_CALLOUT_CALL_TARGET ( const V_MEMRAM1 tFblBmHdrHeader V_MEMRAM2 V_MEMRAM3 * bmHeader )`	
Parameter	
`bmHeader [in]`	FblBmHdrHeader of the software which shall be started.
Return code	
–	-
Functional Description	
Start application target with the specified entry address. This function will never return because control is handed over to the called software instance.	
Particularities and Limitations	
-	
Call context	
-	

Table 6-21 API Description: FBLBM_CALLOUT_CALL_TARGET

6.1.1.21 CALL_FBL_INIT_KEYS

Prototype	
`void FBLBM_CALLOUT_CALL_FBL_INIT_KEYS ( const V_MEMRAM1 tFblBmHdrHeader V_MEMRAM2 V_MEMRAM3 * bmHeader )`	
Parameter	
`bmHeader [in]`	FblBmHdrHeader of the software which shall be started.
Return code	
–	-
Functional Description	
Start Flash Bootloader target with the specified entry address to initialize the keys for the secure boot option. This function will never return because control is handed over to the called software instance.	
Particularities and Limitations	
Only relevant for secure boot.	
Call context	
-	

Table 6-22 API Description: FBLBM_CALLOUT_CALL_FBL_INIT_KEYS

6.1.1.22 CALL_FBL_INIT_CMAC

Prototype	
`void FBLBM_CALLOUT_CALL_FBL_INIT_CMAC ( const V_MEMRAM1 tFblBmHdrHeader V_MEMRAM2 V_MEMRAM3 * bmHeader )`	
Parameter	
`bmHeader [in]`	FblBmHdrHeader of the software which shall be started.
Return code	
–	-
Functional Description	
Start Flash Bootloader target with the specified entry address to calculate and persist the Flash Bootloaders' CMAC for the secure boot option. This function will never return because control is handed over to the called software instance.	
Particularities and Limitations	
Only relevant for secure boot.	
Call context	
-	

Table 6-23 API Description: FBLBM_CALLOUT_CALL_FBL_INIT_CMAC

6.1.1.23 GET_KEYEMPTY

Prototype	
`tFblBmKeyEmptyResult FBLBM_CALLOUT_GET_KEYEMPTY ( void )`	
Parameter	
–	-
Return code	
`FBLBM_KEY_IS_AVAILABLE`	Keys already programmed
`FBLBM_KEY_IS_NOT_AVAILABLE`	No keys programmed
Functional Description	
Checks if keys have been already initialized.	
Particularities and Limitations	
This API is only available if Secure Boot is activated and the Boot Manager is configured to initialize the keys at ECU production time – see Initialize Keys.	
Call context	
-	

Table 6-24 API Description: FBLBM_CALLOUT_GET_KEYEMPTY

6.1.1.24 GET_FBL_CMACERASED

Prototype	

`tFblBmMacEmptyResult` **`FBLBM_CALLOUT_GET_FBL_CMACERASED`**
`(`
`  tFblBmBlockInfo *fblBlockInfo`
`)`	

Parameter	
`fblBlockInfo [in]`	Logical block information structure of Flash Bootloader logical block
Return code	
`FBLBM_MAC_IS_AVAILABLE`	CMAC already programmed
`FBLBM_MAC_IS_NOT_AVAILABLE`	No CMAC available
Functional Description	
Checks if the CMAC for the Flash Bootloader has already been programmed.	
Particularities and Limitations	
This API is only available if Secure Boot is activated and the Boot Manager is configured to initialize the CMAC at ECU production time – see Initialize Mac Of Bootloader	
Call context	
-	

Table 6-25 API Description: FBLBM_CALLOUT_GET_FBL_CMACERASED

6.1.1.25 SEARCH_FAILSAFE_UPDATER_INIT

Prototype	

`tFblResult` **`FBLBM_CALLOUT_SEARCH_FAILSAFE_UPDATER_INIT`**
`(`
`  void`
`)`	

Parameter	
–	-
Return code	
`kFblOk`	Initialization successful
`kFblFailed`	Initialization failed
Functional Description	
Initialize search for Flash Bootloader Updater FblBmHdrHeader location.	
Particularities and Limitations	
This API is only available if Failsafe Updater is activated.	
Call context	
-	

Table 6-26 API Description: FBLBM_CALLOUT_SEARCH_FAILSAFE_UPDATER_INIT

6.1.1.26 SEARCH_FAILSAFE_UPDATER_HEADER_ADDRESS

Prototype	
`tFblResult FBLBM_CALLOUT_SEARCH_FAILSAFE_UPDATER_HEADER_ADDRESS ( tFblAddress *headerAddress )`	
Parameter	
`headerAddress [in]`	Potential FblBmHdrHeader address of Flash Bootloader Updater
Return code	
`kFblOk`	FblBmHdrHeader address could be provided successfully
`kFblFailed`	Failure during FblBmHdrHeader address provision
Functional Description	
Get possible address location of a FblBmHdrHeader for the Flash Bootloader Updater	
Particularities and Limitations	
This API is only available if Failsafe Updater is activated.	
Call context	
-	

Table 6-27 API Description: FBLBM_CALLOUT_SEARCH_FAILSAFE_UPDATER_HEADER_ADDRESS

6.1.1.27 SEARCH_FAILSAFE_UPDATER_NEXT

Prototype	
`tFblResult FBLBM_CALLOUT_SEARCH_FAILSAFE_UPDATER_NEXT ( void )`	
Parameter	
–	-
Return code	
`kFblOk`	Further FblBmHdrHeader address location available
`kFblFailed`	No more FblBmHdrHeader address locations available
Functional Description	
Get next possible address location of a FblBmHdrHeader for the Flash Bootloader Updater	
Particularities and Limitations	
This API is only relevant if the Failsafe Updater option is used.	
Call context	
-	

Table 6-28 API Description: FBLBM_CALLOUT_SEARCH_FAILSAFE_UPDATER_NEXT

6.1.1.28 FATAL_ERROR

Prototype	
`void FBLBM_CALLOUT_FATAL_ERROR ( tFblBmError error )`	
Parameter	
`Error [in]`	Error code (see bm_types.h::tFblBmError)
Return code	
–	-
Functional Description	
General Boot Manager error hook.	
Particularities and Limitations	
-	
Call context	
-	

Table 6-29 API Description: FBLBM_CALLOUT_FATAL_ERROR

6.2 Overview of FblBmHdr Functions

The next section provides the API of the FblBmHdr module.

6.2.1 Core functions

The next section contains a list of the core functions of the FblBmHdr module.

6.2.1.1 FblBmHdrCheckConsistency

Prototype	
`tFblResult FblBmHdrCheckConsistency ( const V_MEMRAM1 tFblBmHdrHeader V_MEMRAM2 V_MEMRAM3 * pBmHeader )`	
Parameter	
`pBmHeader [in]`	Pointer to the FblBmHdrHeader structure
Return code	
`kFblOk`	Consistency check was successful
`kFblFailed`	Consistency check failed
Functional Description	
Checks the consistency of the provided FblBmHdrHeader structure	
Particularities and Limitations	
-	

Call context
-

6.2.1.2 FblBmHdrFindValidHeader

Prototype	

`tFblBmHdrCheckBmHeaderResult` **`FblBmHdrFindValidHeader`**
`(`
`  tFblBmHdrTargetHandle targetHandle,`
`  V_MEMRAM1 tFblBmBlockInfo V_MEMRAM2 V_MEMRAM3 * pBlockInfo`
`)`	

Parameter	
`targetHandle [in]`	Target handle (e.g. Bootloader, Application)
`pBlockInfo [in, out]`	Pointer to a structure which holds a FblBmHdrHeader and the corresponding logical block.
Return code	
`FBLBMHDR_CHKHDR_OK`	A valid BmHeader is found
`FBLBMHDR_CHKHDR_NOT_FOUND`	No BmHeader found
`FBLBMHDR_CHKHDR_BLOCK_INVALID`	Logical block is not valid
`Others`	See return of FblBmHdrCheckBmHeader
`Others`	See return of FblBmHdrVerifyBmHeader
Functional Description	
Find a valid FblBmHdrHeader structure for the given target	
Particularities and Limitations	
-	
Call context	
-	

6.2.1.3 FblBmHdrGetBmHeader

Prototype	

`tFblBmHdrCheckBmHeaderResult` **`FblBmHdrGetBmHeader`**
`(`
`  tFblBmHdrTargetHandle targetHandle,`
`  tFblAddress bmHeaderAddress,`
`  V_MEMRAM1 tFblBmHdrHeader V_MEMRAM2 V_MEMRAM3 * bmHeader`
`)`	

Parameter	
`targetHandle [in]`	Target handle (e.g. Bootloader, Application)

bmHeaderAddress [in]	Location of the FblBmHdrHeader structure
bmHeader [out]	Pointer the FblBmHdrHeader structure
Return code	
FBLBMHDR_CHKHDR_OK	Operation succeded
FBLBMHDR_CHKHDR_READ_FAIL	Reading of the FblBmHdrHeader failed
FBLBMHDR_CHKHDR_HEADER_INCONSISTENT	The FblBmHdrHeader is inconsistence
FBLBMHDR_CHKHDR_WRONG_TARGET	The FblBmHdrHeader is not for the provided targetHandle
Functional Description	
Validate a specific FblBmHdrHeader structure of a logical block	
Particularities and Limitations	
-	
Call context	
-	

6.2.1.4 FblBmHdrGetBmHeader

Prototype	

`tFblBmHdrCheckBmHeaderResult` **`FblBmHdrVerifyBmHeader`**
`(`
`  tFblBmHdrTargetHandle targetHandle,`
`  const V_MEMRAM1 tFblBmBlockInfo V_MEMRAM2 V_MEMRAM3 * pBlockInfo`
`)`	

Parameter	
targetHandle [in]	Target handle (e.g. Bootloader, Application)
pBlockInfo [in, out]	Pointer to a structure which holds a FblBmHdrHeader and the corresponding logical block.
Return code	
FBLBMHDR_CHKHDR_OK	Operation succeeded
FBLBMHDR_CHKHDR_GET_MAC_FAIL	Reading of the MAC failed
FBLBMHDR_CHKHDR_VERIFICATION_FAIL	Verify of the MAC failed
Functional Description	
Verifies a specific FblBmHdrHeader structure of a logical block	
Particularities and Limitations	
> Only available if Secure Boot is enabled	
Call context	
-	

6.2.1.5 FblBmHdrVerifyAreas

Prototype	
`tFblResult FblBmHdrVerifyAreas ( tFblBmHdrTargetHandle targetHandle, const V_MEMRAM1 tFblBmBlockInfo V_MEMRAM2 V_MEMRAM3 * pBlockInfo )`	
Parameter	
`targetHandle [in]`	Target handle (e.g. Bootloader, Application)
`pBlockInfo [in]`	Pointer to a structure which holds a FblBmHdrHeader and the corresponding logical block.
Return code	
`kFblOk`	Operation successful
`kFblFailed`	Operation failed
Functional Description	
Verifies the verification areas specified in the FblBmHdrHeader	
Particularities and Limitations	
> Only available if Secure Boot is enabled	
Call context	
-	

6.2.2 Callback functions

The next sections contains a list of callback functions which are used by the core functions.

6.2.2.1 ApplFblBmHdrCustomVerifyAreasEnable

Prototype	
`tFblResult ApplFblBmHdrCustomVerifyAreasEnable ( tFblBmHdrTargetHandle targetHandle )`	
Parameter	
`targetHandle [in]`	Target handle (e.g. Bootloader, Application)
Return code	
`kFblOk`	Custom verification should be used
`kFblFailed`	Normal verification should be used

Functional Description
Checks if a custom verification for the areas should be used
Particularities and Limitations
> Only available if Secure Boot is enabled and the define FBLBMHDR_ENABLE_CUSTOM_VERIFY_AREAS is set
Call context
-

6.2.2.2 ApplFblBmHdrCustomVerifyAreas

Prototype	
`tFblResult ApplFblBmHdrCustomVerifyAreas ( tFblBmHdrTargetHandle targetHandle, const V_MEMRAM1 tFblBmBlockInfo V_MEMRAM2 V_MEMRAM3 * pBlockInfo )`	
Parameter	
`targetHandle [in]`	Target handle (e.g. Bootloader, Application)
`pBlockInfo [in]`	Pointer to a structure which holds a FblBmHdrHeader and the corresponding logical block.
Return code	
`kFblOk`	Verification was successful
`kFblFailed`	Verification failed
Functional Description	
Perform the custom verification to verify the areas	
Particularities and Limitations	
> Only available if Secure Boot is enabled and the define FBLBMHDR_ENABLE_CUSTOM_VERIFY_AREAS is set	
Call context	
-	

6.3 Data Types

6.3.1 tFblBmHeader

The FblBmHeader contains general information about the Boot Manager.

Type Name	C-Type	Description
`magic`	vuint32	Magic value to mark the start of the FblBmHeader structure.
`majorVersion`	vuint32	Major version of FblBmHeader format

Type Name	C-Type	Description
minorVersion	vuint32	Minor version of FblBmHeader format
fblHeaderAddress	tFblAddress	Pointer to the FBL Header structure, located inside the FBL memory area.
pFblStartFct	tFblStrtFct	Pointer to the FblStart function. This pointer is required in case the feature FblStart is enabled.
pApplStartFct	tApplStrtFct	Pointer to the ApplStart function. This pointer is required in case the feature StayInBoot is enabled.
checksum	vuint32	Checksum of the FblBmHeader structure. This value is currently not used.
end_magic	vuint32	Ensures expected padding.

Table 6-30 FblBmHeader structure definition (tFblBmHeader)

6.3.2 tFblBmHdrHeader

The FblBmHdrHeader structure contains information about the target (e.g. handle) and ranges or an address to the authentication header used for the SecureBoot operation.

Table 6-31 shows the definition in case the FblBmHdrVerificationList is used as the FblBmHdrVariant.

Type Name	C-type	Description
bmMagicFlag	vuint32	Magic value to mark the start of the FblHeader structure.
bmEntryAddress	tFblAddress	Entry address of the target.
bmTargetHandle	tFblBmHdrTarget-Handle	Handle of the target, see FblBmHdr_Cfg.h::tFblBmHdrTargetHandle
bmBlockStartAddress	tFblLength	Start address of the target.
bmBlockLength	tFblLength	Length of the target.
bmVerificationList-Entries	vuint8	Specifies the number of valid entries in bmVerificationList.
bmVerificationList	tFblBmHdrVerification-List	Contains a list of verification ranges used for CMAC verification. A list entry contains a start address and length.

Table 6-31 FblBmHdrHeader structure definition (tFblBmHdrHeader) for FblBmHdrVerificationList

Table 6-32 shows the definition in case the FblBmHdrAuthenticationHeader is used as the FblBmHdrVariant.

Type Name	C-type	Description
bmMagicFlag	vuint32	Magic value to mark the start of the FblHeader structure.
bmEntryAddress	tFblAddress	Entry address of the target.
bmTargetHandle	tFblBmHdrTarget-Handle	Handle of the target, see FblBmHdr_Cfg.h::tFblBmHdrTargetHandle
bmBlockStartAddress	tFblLength	Start address of the target.
bmBlockLength	tFblLength	Length of the target.
bmAuthenticationHeaderAddress	tFblAddress	Address of the authentication header

Table 6-32 FblBmHdrHeader structure definition (tFblBmHdrHeader) for FblBmHdrAuthenticationHeader

6.3.3 tFblBmBlockInfo

Type Name	C-Type	Description
logicalBlock	tBlockDescriptor	Holds a logical block
bmHeader	tFblBmHdrHeader	Holds the corresponding FblBmHdrHeader of the logical block

Table 6-33 FblBmBlockInfo structure definition (tFblBmBlockInfo)

7 Glossary and Abbreviations

7.1 Glossary

Term	Description
Secure Boot	Integrity check at boot time by means of an HTA.
Hardware Trust Anchor	Hardware-based authoritative entity for which trust is assumed and not derived. This can be implemented by means of an HSM or OTP memory.
Flash Bootloader	Software stack to implement a software update capability, e.g. by means of UDS protocol. The Flash Bootloader is typically executed in a separate context.
Flash Bootloader Updater	Application like software to update an existing Flash Bootloader.
Digital Signature	A digital signature provides authenticity and integrity of data. A digital signature algorithm contains both a signature generation process and a signature verification process. The originator uses the signature generation process to generate a signature on data and the users of the data uses the signature verification process to verify a signature on data to ensure the data is authentic.
Integrity	Prevention of unauthorized modification of information.
Boot Target	Execution instance to be started by the Boot Manager, i.e. application, Flash Bootloader, Flash Bootloader Updater.
Dependency Target	Target used as additional dependency check for a Boot Target.
Presence Pattern	Store validity information (block and/or application validity) in flash memory at the respective logical block in case no EEPROM or FEE is available.
Logical Block	Individually erasable/writable software download entity.

7.2 Abbreviations

Abbreviation	Description
AES	Advanced Encryption Standard
CBC	Cipher Block Chaining
CMAC	Block Cipher-based Message Authentication Code
CSM	AUTOSAR Crypto Service Manager
ECB	Encrypt-last-block CBC
FBL	Flash Bootloader
FBT	Flash Block Table
FEE	Flash EEPROM Emulation
HIS	Herstellerinitiative Software
HSM	Hardware Security Module Dedicated microcontroller that safeguards and manages cryptographic keys and secure memory. The HSM creates a secure execution environment for implementing cryptographic primitives.

Abbreviation	Description
HTA	Hardware Trust Anchor
LBT	Logical Block Table
NVM	Non Volatile Memory, e.g. EEPROM, FEE
OTP	One-time programmable
SHE	Secure Hardware Extension
vHsm	Vector HSM-based HTA solution

8 Contact

Visit our website for more information on

- News
- Products
- Demo software
- Support
- Training data
- Addresses

www.vector.com